MAYA Verifier Specification

Version 2.0
August 25, 2026

Thi Van Thao Doan, Olivier Pereira, and Thomas Peters
UCLouvain, Belgium

Contents

1 Introduction

In an end-to-end verifiable election, encrypted ballots are published on a bulletin board. To tally without revealing individual votes, one or more *mix servers* each take a list of n ElGamal ciphertexts, secretly permute them, and re-randomize each ciphertext so that the output list is unlinkable to the input. A *proof of shuffle* convinces any observer that the mix server did not add, drop, or alter any vote, only reorder and re-randomize. MAYA [1] is such a proof, with $O(\log n)$ proof size and fast verification.

This document provides a complete, language-agnostic verification procedure for MAYA. Together with the election record (Section 4.2), this specification enables an independent developer to write a working verifier from scratch, without access to the MAYA source code [2] or the research paper [1].

1.1 Scope and Requirements

This is a *verifier-only* specification. It defines the exact binary format of all data (Section 4), the deterministic derivation of public parameters (Section 5), the hash-based Fiat–Shamir transcript protocol (Section 6), and the step-by-step verification algorithm with explicit bounds checks (Section 8). It does *not* describe the prover's construction (see [1]).

Byte-level reproducibility. Two conforming implementations in different languages must produce identical accept/reject decisions on identical input bytes. Every array length is stated and must be verified; every scalar and group element is checked for canonicality. Equation and section numbers from the MAYA paper [1] (e.g. "Eq. 25", "Appendix F") appear throughout only as informative cross-references; every formula required to implement the verifier is restated in full in this document.

1.2 Roadmap

Section 2 gives a high-level overview of the MAYA protocol and its verification pipeline. Section 3 defines the group and scalar arithmetic. Section 4 defines how to read bytes into the objects from Section 3. Section 5 derives the public generators (CRS) using the hash-to-group map from Section 3 and the encoding conventions from Section 4. Section 6 defines the HMAC-based Fiat–Shamir transcript. Section 7 derives the challenge vector $\boldsymbol{e}$ using the dedicated transcript (Section 6) and the generators from Section 5. Section 8 gives the complete verification algorithm in six numbered steps. The IPA and eCP sub-protocols used by the verifier are described separately in Sections 9 and 10 to improve readability and modularity. Section 11 provides test vectors for each intermediate computation.

1.3 Implementation Steps

The verifier can be built and tested incrementally. Each step below produces a testable output that can be validated against the test vectors in Section 11 before moving to the next.

Step	Task	What to test	Section
1	**Scalar and point I/O.** Implement scalar encoding/decoding, point compression/decompression, and canonicality checks.	Decode test-vector scalars and points; confirm round-trip.	3, 4.1

2	**CRS generators.** Implement the HMAC-SHA-256 counter mode, hash-to-group map (FromUniformBytes), and the Pedersen / Bulletproof generator derivation.	Compare B_{blinding}, F, first 4 entries of $\boldsymbol{g}$ and $\boldsymbol{h}$ against test vectors.	5
3	**HMAC transcript.** Implement the HMAC-SHA-256 transcript state machine (init, append, challenge).	Replay the test-vector transcript and compare each derived challenge $(y, z, x, \ldots)$.	6
4	**Challenge vector $\boldsymbol{e}$.** Implement the dedicated transcript and counter-mode HMAC expansion.	Compare first 4 entries of $\boldsymbol{e}$ against test vectors.	7
5	**Election record parser.** Parse the input ciphertexts file and the proof bundle; run all structural validation checks.	Parse the test-vector files; confirm n, k, point counts.	4
6	**Constraint flattening.** Build the permutation circuit and compute the weight vectors $\boldsymbol{w}_L, \boldsymbol{w}_R, \boldsymbol{w}_O, \boldsymbol{w}_V, w_c$.	Compare against the flattened-weight test vectors for $n = 4$.	8 (Verification 3)
7	**IPA scalar expansion.** Implement the IPA verification-scalars function: replay the IPA sub-transcript, expand the final witness back to n dimensions.	(No separate test vector; correctness is checked end-to-end in step 9.)	9
8	**eCP scalar expansion.** Same as step 7 for the eCP sub-protocol.	(End-to-end in step 9.)	10
9	**Mega-MSM assembly.** Compute all batched scalars, assemble the point/scalar vectors, execute the MSM, and check identity.	Run the complete verifier on the test-vector proof. Expected result Accept.	8 (Verifications 4–6), 8.6

1.4 Notation

We write $\mathbb{Z}_p$ for the field of integers modulo a prime p, $\mathbb{Z}_p^n$ for the n-dimensional vector space over $\mathbb{Z}_p$, and $\mathbb{G}^n$ for n-tuples of elements in a cyclic group $\mathbb{G}$ of order p. Bold letters denote vectors $\boldsymbol{a} = (a_1, \ldots, a_n) \in \mathbb{Z}_p^n$. Given two vectors $\boldsymbol{a}, \boldsymbol{b} \in \mathbb{Z}_p^n$, the *inner product* is $\langle \boldsymbol{a}, \boldsymbol{b} \rangle = \sum_{i=1}^n a_i b_i \in \mathbb{Z}_p$, and the *Hadamard* (component-wise) product is $\boldsymbol{a} \circ \boldsymbol{b} = (a_1 b_1, \ldots, a_n b_n) \in \mathbb{Z}_p^n$. For $\boldsymbol{g} = (g_1, \ldots, g_n) \in \mathbb{G}^n$ and $\boldsymbol{a} \in \mathbb{Z}_p^n$, we write $\boldsymbol{g}^{\boldsymbol{a}} = \prod_{i=1}^n g_i^{a_i} \in \mathbb{G}$ for the multi-scalar multiplication (MSM). The symbol $\mathcal{S}_n$ denotes the symmetric group on $[n] = \{1, \ldots, n\}$. We adopt multiplicative group notation throughout, following [1].

2 The MAYA Shuffle Argument

This section introduces the MAYA protocol from the verifier's perspective. It explains what statement a MAYA proof establishes, how the protocol is structured, and how its main cryptographic components fit together. The detailed definitions, encodings, and verification procedures are specified in the sections that follow.

2.1 Shuffle Statement

Let $(\mathbb{G}, p, g)$ be a cyclic group of prime order p. Given n ElGamal ciphertexts $C_i = (C_{i,1}, C_{i,2})$ under public key $\text{pk} = (g, f)$ with $f = g^x$ for a secret key x shared among trustees, a mix server produces output ciphertexts

$$C'_i \;=\; C_{\pi(i)} \cdot (g, f)^{r_i} \;=\; \big(C_{\pi(i),1} \cdot g^{r_i},\; C_{\pi(i),2} \cdot f^{r_i}\big), \tag{1}$$

for a secret permutation $\pi \in \mathcal{S}_n$ and re-randomization scalars $\boldsymbol{r} \in \mathbb{Z}_p^n$. The verifier must be convinced of this claim without learning π or $\boldsymbol{r}$.

Formal statement. The verifier **accepts** if and only if it is convinced that the prover knows a witness for the shuffle relation (Eq. 3 in [1]):

$$\mathcal{R}_{\text{shuffle}} = \Big\{ \big(\text{pk}, \boldsymbol{C}, \boldsymbol{C}';\ \pi, \boldsymbol{r}\big) : \pi \in \mathcal{S}_n,\ \boldsymbol{r} \in \mathbb{Z}_p^n,\ C'_i = C_{\pi(i)} \cdot (g, f)^{r_i}\ \forall\, i \in [n] \Big\}. \tag{2}$$

Everything in Sections 3–10 exists to decide this single statement.

Reduction to two claims. Following Wikström [3], the verifier derives a random challenge vector $\boldsymbol{e} = (e_1, \ldots, e_n) \in \mathbb{Z}_p^n$ by hashing all public data (Section 7). Since $\boldsymbol{e}$ is determined by the ciphertext lists themselves, neither the prover nor the verifier can choose it. Let $\boldsymbol{e}' = (e_{\pi(1)}, \ldots, e_{\pi(n)})$ denote the permuted challenges, and let $r' = \langle \boldsymbol{e}', \boldsymbol{r} \rangle$ be the aggregated re-randomization scalar. The shuffle correctness (2) reduces to checking two claims:

(i) Permutation: $\boldsymbol{e}'$ is a permutation of $\boldsymbol{e}$.

(ii) Ciphertext consistency: The aggregated ciphertext equations hold:

$$g^{-r'} \cdot \boldsymbol{c}_1'^{\,\boldsymbol{e}'} = \boldsymbol{C}_1^{\boldsymbol{e}}, \qquad f^{-r'} \cdot \boldsymbol{c}_2'^{\,\boldsymbol{e}'} = \boldsymbol{C}_2^{\boldsymbol{e}}, \tag{3}$$

where $\boldsymbol{c}'_k = (C'_{1,k}, \ldots, C'_{n,k})$ and $\boldsymbol{C}_k^{\boldsymbol{e}} = \prod_i C_{i,k}^{e_i}$ for $k \in \{1, 2\}$.

All quantities in (3) are public except $\boldsymbol{e}'$ and r'. The prover commits $\boldsymbol{e}'$ in a single Pedersen vector commitment

$$V \;=\; h^s \cdot \boldsymbol{g}^{\,\boldsymbol{e}'}, \tag{4}$$

where $\boldsymbol{g} = (g_1, \ldots, g_n) \in \mathbb{G}^n$ and $h \in \mathbb{G}$ are independent generators from a common reference string (CRS) with no known discrete-log relations (Section 5). The protocol then proves both claims in two phases (Sections 2.3–2.4) that share the commitment V, ensuring that the same permutation underlies both.

2.2 Building Blocks

Before describing the two phases, we briefly explain the two proof primitives that MAYA uses to achieve logarithmic proof size.

Inner product argument (IPA). An inner product argument allows a prover to demonstrate knowledge of vectors $\boldsymbol{a}, \boldsymbol{b} \in \mathbb{Z}_p^n$ such that a commitment $P = \boldsymbol{g}^{\boldsymbol{a}}\, \boldsymbol{h}^{\boldsymbol{b}}\, u^{\langle \boldsymbol{a}, \boldsymbol{b} \rangle}$ opens correctly, without revealing $\boldsymbol{a}$ or $\boldsymbol{b}$ (here $\boldsymbol{g}, \boldsymbol{h} \in \mathbb{G}^n$ are independent generator vectors and $u \in \mathbb{G}$ is a further independent generator) [4, 5]. The key idea is *folding*: the prover splits each witness vector into k blocks (k-ary folding), computes $2(k-1)$ cross-term commitments, and the verifier replies with a random challenge. Both parties then combine the blocks into a new vector of size n/k, preserving the algebraic structure. After $d = \lceil \log_k n \rceil$ rounds, only a constant-size base-case witness remains. Each round transmits $2(k-1)$ group elements, so the total proof consists of $2(k-1)\log_k n$ group elements plus $O(1)$ scalars.

Extended Chaum–Pedersen argument (eCP). The eCP is a compressed Σ-protocol [6] for proving a linear-map preimage relation $\boldsymbol{P} = \boldsymbol{G}^{\boldsymbol{a}}$, where $\boldsymbol{G} \in \mathbb{G}^{\gamma \times n}$ is an arbitrary public matrix, witness $\boldsymbol{a} \in \mathbb{Z}_p^n$, and $\boldsymbol{P} \in \mathbb{G}^\gamma$ is a target vector [7]. A direct Σ-protocol [8] for this relation would require transmitting $\mathbf{a}$ (or a masked version of it) as an n-element response vector. The eCP instead applies the same k-ary folding technique as the IPA to compress the response. For $\gamma = 2$ (as used throughout this specification), the proof size is comparable to the IPA cost.

Iterative padding. Both the IPA and eCP require the witness dimension to be divisible by k at each recursion step. When this does not hold, MAYA applies *iterative padding* [1]: at each round, if the current dimension n_r satisfies $n_r \not\equiv 0 \pmod{k}$, the prover appends $k - (n_r \bmod k)$ zero entries to the witness and correspondingly extends the generator vectors. Over $d = \lceil \log_k n \rceil$ rounds the total padding overhead is bounded by $(k-1) \cdot d$ elements. The verifier reconstructs round dimensions using Algorithm 1 in Section 4.6.

2.3 Phase 1: Permutation Argument

Phase 1 proves claim (i): that $\boldsymbol{e}'$ is a permutation of $\boldsymbol{e}$. Two vectors contain the same multiset of values if and only if the polynomials $\prod_i (e'_i - X)$ and $\prod_i (e_i - X)$ are identical. By the Schwartz–Zippel lemma, it suffices to check at a single random challenge point $u \in \mathbb{Z}_p$, derived from the transcript (Section 8, step 3.1):

$$\prod_{i=1}^{n} (e'_i - u) \;=\; \prod_{i=1}^{n} (e_i - u). \tag{5}$$

MAYA encodes this product as an arithmetic circuit with $n-1$ chained multiplication gates: gate 1 computes $(e'_1 - u)(e'_2 - u)$, and each subsequent gate multiplies the running product by the next factor $(e'_{j+1} - u)$. The circuit is expressed in the R1CS (Rank-1 Constraint System) framework of Bulletproofs [5] with $Q = 2n-1$ linear constraints. Specifically, the R1CS encoding is defined by public coefficient matrices $(\boldsymbol{w}_L, \boldsymbol{w}_R, \boldsymbol{w}_O, \boldsymbol{w}_V)$ and a constant vector $\boldsymbol{c}$, which specify how gate inputs and outputs are connected to each other and to the committed witness $\boldsymbol{e}'$ [5]. The resulting relation is proved via an IPA.

Because $\boldsymbol{e}'$ lives in a single vector commitment V rather than in n individual commitments, the verifier cannot directly evaluate the linear term that appears in the R1CS verification equation. MAYA resolves this through an auxiliary bridging commitment [1, Section 3.2]: a second IPA instance that certifies the witness committed in V matches the circuit's linear constraints. The challenge x_{ipp} aggregates the circuit-satisfiability statement with the witness-binding statement into a single IPA instance.

At the end of Phase 1 the verifier is convinced that V commits to *some* permutation of the public vector $\boldsymbol{e}$.

2.4 Phase 2: Ciphertext Consistency

Phase 2 proves claim (ii): that the same $\boldsymbol{e}'$ committed in V satisfies the aggregated ciphertext equations (3). A standard Σ-protocol [8] could prove this in zero knowledge by transmitting a masked n-dimensional witness $\boldsymbol{l}_c = \boldsymbol{e}' + \boldsymbol{s}'_L \cdot x'$ (where $\mathbf{s}'_L \xleftarrow{\$} \mathbb{Z}_p^n$ and $x' \xleftarrow{\$} \mathbb{Z}_p$), but this costs $O(n)$ communication. MAYA instead uses the eCP technique. The three verification equations are formulated as a $3 \times n$ matrix $\boldsymbol{G}$ with rows $(\boldsymbol{g}, \boldsymbol{c}'_1, \boldsymbol{c}'_2)$. Using random linear combinations via a challenge α_{ecp}, the two ciphertext rows are batched into a single row, yielding a 2-row eCP instance. The commitment row $\boldsymbol{g}$ is kept separate: since its generators are independently sampled, it forms a binding commitment that fixes $\boldsymbol{l}_c$ before the batching challenge is generated. In contrast, the ciphertext rows involve bases whose discrete logarithm relations may be known to the prover; batching all rows into one equation would allow a malicious prover to satisfy the combined relation without satisfying each individually [7].

Cross-Phase Binding. Both phases share the same commitment V, the same masking vector $\boldsymbol{s}'_L$, and thus the same evaluated witness $\boldsymbol{l}_c = \boldsymbol{e}' + \boldsymbol{s}'_L \cdot x'$. This prevents a cheating prover from independently tailoring separate witnesses for the permutation and ciphertext checks. Any inconsistency would require breaking the binding property of V under the Discrete Logarithm Relation (DLR) assumption [4].

Batched verification. The verifications above reduce to a handful of group equations, each of the form $\sum_j s_j P_j = \mathcal{O}$. Rather than checking them independently, the verifier scales each equation by a power of a deterministic batching scalar r (derived from the transcript after all proof data has been committed) and sums them into a single multi-scalar multiplication—the *Mega-MSM* (Section 8.6.2). The five equation families (IPA folding, Phase-1 polynomial check, bridging polynomial check, eCP ciphertext equation, eCP commitment equation) are combined using r^0, r^1, r^2, r^3, r^4 respectively. By the Schwartz–Zippel lemma, if any individual equation fails, the combined MSM evaluates to the identity with probability at most $4/p < 2^{-250}$. Section 8.6 details what each power of r carries.

The resulting proof consists of one batched IPA and one 2-row eCP, both over n-dimensional vectors, with total proof size $O(\log n)$ group elements.

2.5 Protocol Objects

The table below inventories every object that appears in the protocol, who knows it, and where this document defines its encoding or derivation.

Object	Known to	Description	Defined in
$\mathsf{pk} = (g, f)$	Public	ElGamal public key	§4.2
h	Public	Pedersen blinding base	§4.2
$\boldsymbol{C}$	Public	Input ciphertext list (n pairs)	§4.2
$\boldsymbol{C}'$	Public	Output ciphertext list (n pairs)	§4.2
$\boldsymbol{g}, \boldsymbol{h}, B, B_{\text{blinding}}, F$	Publicly derived	CRS generators (recomputed, never read from input)	§5
$\boldsymbol{e}$	Publicly derived	Challenge vector	§7
y, z, u, x, x', etc.	Publicly derived	Fiat–Shamir challenges	§6, §8
π	Prover only	Secret permutation	—
$r_1, \ldots, r_n$	Prover only	Re-randomization scalars	—
$\boldsymbol{e}'$, s	Prover only	Permuted challenges, V's blinding	—
V	Public (proof)	Vector commitment to $\boldsymbol{e}'$	§4.2
π_{proof}	Public (proof)	Header points/scalars, IPA and eCP sub-proofs	§4.3

2.6 Trust Model and Verification Scope

Soundness of the verification rests on:

1. The hardness of the Discrete Logarithm Relation (DLR) problem in the group $\mathbb{G}$ of Section 3 (for polynomially bounded n, the DLR assumption is equivalent to the standard discrete logarithm assumption [4]);
2. HMAC-SHA-256 behaving as a random oracle for the Fiat–Shamir transformation (the security proofs in [1] are hash-agnostic in the random-oracle model); and
3. HMAC-SHA-256, used in the counter mode of Section 5, producing CRS generators with no discrete-log relations known to any party.

Every byte of the election record and the proof is treated as potentially adversarial: a malicious prover may submit arbitrary byte strings. This is why the specification mandates explicit bounds checks, canonical scalar checks, and decompression checks on every field. The verifier must recompute the CRS generators and all challenges itself. It must not accept generators, challenge vectors, or challenge scalars supplied in the input data.

Out of scope. The following are deliberately outside this document: the generation and authenticity of the ElGamal public key (g, f) (established by a trustee key ceremony and the surrounding election system); the validity of the plaintext votes; the decryption of the final ciphertext list and its proofs; and the consistency of the bulletin board. A MAYA proof only shows that $\boldsymbol{C}'$ is a correct shuffle of $\boldsymbol{C}$ for the given public key.

2.7 Verification Pipeline

A conforming verifier executes the following six steps in order. This table serves as a roadmap. All variables referenced below are defined earlier in this section or in the indicated sections.

Step	Action
Verification 1 (§8.1)	Parse and validate the election record (Section 4.2). Check exact file lengths, $n \geq 2$, and $k \in \{2, 3, 4, 5, 6, 8, 16\}$. Decompress all points; reject non-canonical scalars. Derive CRS generators (Section 5) and challenge vector $\boldsymbol{e}$ (Section 7).
Verification 2 (§8.2)	Replay the Fiat–Shamir transcript (Section 6) to derive all challenges. Reject if any challenge is zero.
Verification 3 (§8.3)	Derive the shuffle challenge u (via the transcript label `"shuffle challenge"`). Build the permutation circuit's weight vectors $\boldsymbol{w}_L, \boldsymbol{w}_R, \boldsymbol{w}_O \in \mathbb{Z}_p^{n-1}$, $\boldsymbol{w}_V \in \mathbb{Z}_p^{n_{\text{pad}}}$, $w_c \in \mathbb{Z}_p$ (Section 8).
Verification 4 (§8.4)	Run the IPA verification-scalars sub-protocol (Section 9): replay the IPA folding rounds to obtain $(\boldsymbol{s}_g, \boldsymbol{s}_h, s_Q, s_P, \boldsymbol{s}_U)$. Reject if any round challenge is zero or any cross-term point fails decompression.
Verification 5 (§8.5)	Derive α_{ecp} and run the eCP verification-scalars sub-protocol (Section 10). Derive the deterministic batching scalar r (transcript label `"batching_r"`). Compute all combined scalars for the Mega-MSM, including the five batched equation families (Section 8.6).
Verification 6 (§8.7)	Execute the Mega-MSM: $R = \sum_j s_j P_j$. *Accept* if and only if $R = \mathcal{O}$ (the group identity). Otherwise *reject*.

The entire verification is deterministic: given the same election record and proof bytes, any implementation in any language produces the same accept/reject decision.

3 Groups and Arithmetic

MAYA is defined over an arbitrary prime-order group $\mathbb{G}$ of order p with generator g. The protocol is algebraically generic: the proof system, the Fiat–Shamir transcript, the CRS derivation, and the verification equations are all stated in terms of abstract group operations. The specification is therefore *parametric* in the choice of group; only the concrete byte-level encodings, the hash-to-group map, and certain byte-order conventions depend on the instantiation.

A group instantiation must provide the following four operations:

Compress$(P) \to \mathcal{B}^c$**:** Serialize a group element to a fixed-length byte array of c bytes.

Decompress$(b) \to P \mid \perp$**:** Deserialize a byte array to a group element, or fail. The function must reject non-canonical encodings.

IsIdentity$(P) \to \{\text{true}, \text{false}\}$**:** Check whether P is the identity element.

FromUniformBytes$(b \in \mathcal{B}^{64}) \to P$**:** Map 64 uniform random bytes to a group element. This is the hash-to-group primitive used exclusively for deriving CRS generators (Section 5).

This specification standardizes a Ristretto255 instantiation of MAYA. All normative verification algorithms, examples, and test vectors are defined with respect to this parameter set.

3.1 Ristretto255

Ristretto255, specified in RFC 9496, is the primary instantiation of MAYA [1] and the parameter set used by Version 2 of the election record format (Section 4.2). It provides a prime-order abstraction over Curve25519 and supports all four operations listed above. The essential parameters are:

Parameter	Value
Group identifier	"ristretto255"
Underlying curve	Curve25519 (RFC 7748)
Scalar field order	$p = 2^{252} + 27742317777372353535851937790883648493$
Compressed element size	32 bytes
Scalar size	32 bytes (little-endian canonical)
Identity encoding	32 zero bytes
Byte order	Little-endian for both scalars and compressed points

The full set of hexadecimal constants, including the basepoint encoding, the scalar field order in hex, and the curve equation, is given in Appendix A for implementers who need to hard-code these values.

Compress & Decompress. Ristretto255 defines a canonical compressed encoding (RFC 9496, §3.3.3–3.3.4). Every valid Ristretto point has exactly one 32-byte compressed form. Decompress must reject non-canonical encodings.

Implementation note. The identity element has canonical compressed encoding `0x00...00` (32 zero bytes) and is a valid Ristretto255 group element. Some libraries treat it specially: for example, `libsodium`'s `crypto_core_ristretto255_is_valid_point` returns 0 (invalid) for the identity, and `crypto_scalarmult_ristretto255` returns -1 for scalar 0. Implementations using such libraries must (a) check for the identity encoding before calling the validity function, and (b) skip zero-scalar MSM terms rather than passing them to the scalar-multiplication function. These are library-specific behaviors, not protocol requirements. Implementations using `curve25519-dalek` (Rust), `filippo.io/edwards25519` (Go), or other libraries may not encounter these issues. The protocol requires only that Decompress(`0x00...00`) succeeds and returns the identity, and that IsIdentity($\mathcal{O}$) = true.

Scalar encoding. Scalars are 256-bit integers in $[0, p)$, encoded as 32-byte little-endian arrays (least significant byte first). A scalar s is canonical if and only if $s < p$.

FromUniformBytes. The map FromUniformBytes : $\mathcal{B}^{64} \to \mathbb{G}$ is defined in RFC 9496, §5.3. It splits the 64 input bytes into two 32-byte halves, maps each half to a curve point via the Elligator2 map, and adds the two curve points. For implementers writing this function from scratch (rather than calling a library), the step-by-step algorithm is provided in Appendix C. Most languages have a Ristretto255 library that already exposes this function (e.g. `curve25519-dalek` in Rust, `ristretto255-js` in JavaScript, `filippo.io/edwards25519` in Go).

Implementation note. A Ristretto255 library for a MAYA verifier must expose (directly or via wrappers): Compress ($P \to$ 32 bytes), Decompress (32 bytes $\to P$ or fail), IsIdentity ($P \to$ bool), FromUniformBytes (64 bytes $\to P$), scalar multiplication ($s, P \to P$), point addition ($P, Q \to P$), and scalar-field arithmetic (add, subtract, multiply, invert modulo p). Additionally, HMAC-SHA-256 from a standard cryptographic library is required (Section 6.1).

3.2 Other Groups

The MAYA protocol is not tied to Ristretto255. The academic paper [1] provides benchmarks for a secondary instantiation over NIST P-256 (secp256r1), and the protocol applies equally to any prime-order group satisfying the Discrete Logarithm Relation (DLR) assumption.

A future version of the election record format may introduce additional group instantiations. Each new instantiation must specify:

1. the FromUniformBytes map (e.g. RFC 9380 hash-to-group for P-256);
2. the point compression/decompression rules and element size c;

3. the scalar encoding and byte order;
4. any adjustments to the CRS derivation (Section 5).

A verifier conforming to this specification (Version 2) must use the Ristretto255 parameter set when verifying a election record (Section 4.2).

3.3 Implementation Notes

A MAYA verifier performs five categories of operations.

Scalar field arithmetic. Scalars are integers in $\mathbb{Z}_p = \{0, 1, \ldots, p-1\}$. Addition, subtraction, and multiplication are performed modulo p. Inversion $a^{-1} \bmod p$ is computed by extended Euclidean algorithm or by Fermat's little theorem ($a^{-1} = a^{p-2} \bmod p$). All intermediate results must be reduced modulo p before use.

Scalar multiplication. Given a scalar $s \in \mathbb{Z}_p$ and a group element $P \in \mathbb{G}$, the scalar multiplication $s \cdot P$ (also written P^s in multiplicative notation) is the core operation. It should be implemented using a constant-time double-and-add algorithm or a windowed method.

Multi-scalar multiplication (MSM). The MSM $R = \sum_{i=1}^{N} s_i \cdot P_i$ computes a linear combination of N points with N scalars. This is the dominant computation in MAYA verification. Efficient algorithms include Pippenger's bucket method and Straus's multi-exponentiation. A verifier may use variable-time MSM algorithms since all inputs are public.

Point decompression. Each group element must be decompressed via Decompress before use. If decompression fails, the proof must be rejected.

Hash computation. A single hash primitive is used throughout this specification: HMAC-SHA-256 (NIST FIPS 198-1/180-4). It was chosen because it is the most widely available keyed hash and produces deterministic 32-byte outputs with domain separation, making every derived value byte-exact across languages. It serves three roles (see Section 6 for details):

1. the Fiat–Shamir transcript (Section 6.3);
2. deterministic generator derivation (CRS, Section 5), where two counter-indexed calls produce the 64 uniform bytes consumed by FromUniformBytes;
3. challenge-vector expansion (Section 7), using the same two-call counter mode.

HMAC-SHA-256 is available in the standard libraries of C (OpenSSL), Python (`hashlib`, `hmac`), Go, Java (JDK 9+), and Rust.

4 Data Formats and Serialization

This section defines how to parse bytes into well-typed objects and which structural checks to apply. It is purely syntactic: the meaning of each object is given in the protocol overview (Section 2) and the object inventory (Section 2.5); the role each proof element plays in the verification equations is specified in Sections 8–10. Implementers may find it useful to write and test a parser for this section against the test vectors (Section 11) before implementing any cryptographic logic.

All integers are encoded in *little-endian* byte order. All sizes below are for Ristretto255 (32-byte elements); for other group instantiations (Section 3.2), replace 32 with the group's compressed element size c.

4.1 Encoding Conventions

All inputs to the hash function H and all serialized data are byte arrays. A byte is a non-negative integer in $\mathcal{B} = \{0, 1, \ldots, 255\}$; $\mathcal{B}^m$ denotes byte arrays of length m, and $\mathcal{B}^*$ denotes all finite-length byte arrays.

Scalars. A scalar $s \in \mathbb{Z}_p$ is encoded as a 32-byte array in canonical little-endian form:

$$\mathsf{b}(s) = s_0 \| s_1 \| \cdots \| s_{31}, \quad \text{where } s_i = \lfloor s/256^i \rfloor \bmod 256. \tag{6}$$

The verifier must verify that every decoded scalar satisfies $0 \leq s < p$. For Ristretto255, any 32-byte array with $s_{31} \geq$ `0x10` is immediately non-canonical; otherwise, the decoded integer must be compared against p.

Group elements. A group element $P \in \mathbb{G}$ is encoded as a 32-byte compressed Ristretto255 point (RFC 9496, §3.3.3). The verifier must call Decompress on every 32-byte group element; if decompression fails, the proof is rejected immediately.

Small integers. Small integers are encoded as fixed-length little-endian unsigned integers (least significant byte first). Two widths are used:

$$\mathsf{b}_8(n) = n_0 \| n_1 \| \cdots \| n_7 \qquad \text{(8 bytes, u64)}, \tag{7}$$

$$\mathsf{b}_4(n) = n_0 \| n_1 \| n_2 \| n_3 \qquad \text{(4 bytes, u32)}, \tag{8}$$

where $n_i = \lfloor n/256^i \rfloor \bmod 256$ in both cases. The 8-byte form b_8 is used for array lengths, folding factors, and round counts. The 4-byte form b_4 is used for the length prefix inside transcript messages (Eq. 11) and for the index counter in the challenge-vector expansion (Section 7). For example, $\mathsf{b}_4(5) =$ `0x05 0x00 0x00 0x00` and $\mathsf{b}_8(5) =$ `0x05 0x00 0x00 0x00 0x00 0x00 0x00 0x00`.

String labels. String labels used in the transcript (e.g. `"A_I"`, `"dom-sep"`) are encoded as raw UTF-8 bytes without a length prefix.

4.2 Verifier Input

In the reference implementation, the verifier reads two inputs:

1. Input ciphertexts file (from the bulletin board or the previous mix layer): contains the number of ciphertexts n, the two coordinate vectors $\boldsymbol{c}_1, \boldsymbol{c}_2$ (n points each), and the public key (g, f, h).
2. Proof bundle (from the prover): contains the serialized proof π, the vector commitment V, and the output ciphertext coordinate vectors $\boldsymbol{c}'_1, \boldsymbol{c}'_2$ (n points each).

The verifier reads two physical files. File 1, the *input ciphertexts file* (reference name `maya_ristretto_bench_input_ciph.bin`), contains the fields of the first table below, starting at offset 0. File 2, the *proof bundle* (reference name `maya_ristretto_bench_output_bundleh.bin`), contains the fields of the second table, also starting at offset 0. An implementation may source these fields from a database or network API instead, as long as every field is available and every structural check of Verification 1 is applied. Each file must have exactly the total length implied by its declared counters (checks (1.D), (1.N)); trailing bytes are rejected.

Offset	Field	Size	Description
0	n	8 B	u64 LE: number of ciphertexts
8	$\boldsymbol{c}_1$	$32n$ B	Input ciphertext first components
8+32n	$\boldsymbol{c}_2$	$32n$ B	Input ciphertext second components
8+64n	g	32 B	ElGamal generator
40+64n	f	32 B	ElGamal public key
72+64n	h	32 B	Pedersen blinding base
— proof bundle (from prover) —			
	proof_len	8 B	u64 LE: serialized proof byte length
	proof	ℓ B	Serialized R1CSProof (Section 4.3)
	V	32 B	Vector commitment to $\boldsymbol{e}'$
	n'	8 B	u64 LE: number of output ciphertexts (must equal n)
	$\boldsymbol{c}'_1$	$32n$ B	Output ciphertext first components
	n''	8 B	u64 LE: number of output ciphertexts (must equal n)
	$\boldsymbol{c}'_2$	$32n$ B	Output ciphertext second components

The structural checks on these fields are collected in Verification 1 (Section 8.1).

4.3 Proof Format (`R1CSProof`)

The serialized proof consists of a fixed-size header followed by variable-length IPA and eCP sub-proofs. The header elements are: A_I, A_O, S (Phase-1 wire commitments), $T_1, \ldots, T_6$ (polynomial coefficients; T_2 is the bridging commitment, see Section 2.3), S', T'_1, S'_1, S'_2 (Phase-2 commitments), and eight evaluation scalars ($t_x, \tau_x, \mu, t_{c,x}, \tau_{c,x}, \mu_c, t_{\text{cross}}, \rho_c$). The table below is the fixed header (672 bytes).

Offset	Field	Type	Size	Transcript label (§6.4)
0	A_I	Point	32 B	`A_I`
32	A_O	Point	32 B	`A_0`
64	S	Point	32 B	`S`
96	T_1	Point	32 B	`T_1`
128	T_2	Point	32 B	`T_2`
160	T_3	Point	32 B	`T_3`
192	T_4	Point	32 B	`T_4`
224	T_5	Point	32 B	`T_5`
256	T_6	Point	32 B	`T_6`
288	S'	Point	32 B	`S_prime`
320	T'_1	Point	32 B	`T_1_prime`
352	S'_1	Point	32 B	`S1_prime`
384	S'_2	Point	32 B	`S2_prime`
416	t_x	Scalar	32 B	`t_x`
448	τ_x	Scalar	32 B	`t_x_blinding`
480	μ	Scalar	32 B	`e_blinding`
512	$t_{c,x}$	Scalar	32 B	`tc_x`
544	$\tau_{c,x}$	Scalar	32 B	`tc_x_blinding`
576	μ_c	Scalar	32 B	`ec_blinding`
608	ρ_c	Scalar	32 B	`r_blinding`
640	t_{cross}	Scalar	32 B	`t_cross`
672	ℓ_{IPA}	u64	8 B	–
680	ℓ_{eCP}	u64	8 B	–
688	IPA data	bytes	ℓ_{IPA}	(§B.3)
$688 + \ell_{\text{IPA}}$	eCP data	bytes	ℓ_{eCP}	(§B.4)

Validation (Verification 1, part b).

(a) Total proof length must equal $688 + \ell_{\mathsf{IPA}} + \ell_{\mathsf{eCP}}$.
(b) All 13 points must decompress. All 8 scalars must be canonical ($< p$).
(c) The IPA sub-proof must contain exactly $d \cdot (2k - 2)$ cross-term points, where k and d are read from the IPA header below.
(d) The eCP sub-proof must contain exactly $d_e \cdot (2k_e - 2)$ cross-term point *pairs*.

4.4 IPA Sub-Proof Format

The IPA sub-proof begins with three parameters, each stored as a u64 in a 32-byte slot (8 bytes of value followed by 24 bytes of zero padding), followed by the cross-term commitments and the final witness vectors. These are the folding proof data used in Verification 4 (Section 9).

Field	Size	Content
k	32 B	Folding factor (u64 in first 8 bytes, rest zero)
d	32 B	Number of folding rounds
m	32 B	Base-case witness size
$\{U_r\}$	$d \cdot (2k-2) \cdot 32$ B	Cross-term points (round-major order)
$\boldsymbol{a}_f$	$m \cdot 32$ B	Final left witness scalars
$\boldsymbol{b}_f$	$m \cdot 32$ B	Final right witness scalars

The verifier must confirm that $|\boldsymbol{a}_f| = |\boldsymbol{b}_f| = m$, where m is recomputed from Algorithm 1 (Section 4.6).

4.5 eCP Sub-Proof Format

The eCP sub-proof has the same 32-byte-slot header (k, d, m) as the IPA. Cross-terms are *pairs* of points (one pair per folding block), written as two consecutive 32-byte compressed points. These are the folding proof data used in Verification 5 (Section 10).

Field	Size	Content
k, d, m	3×32 B	Same layout as IPA
$\{A_r\}$	$d \cdot (2k-2) \cdot 64$ B	Cross-term point pairs $[A^{(0)}, A^{(1)}]$
$\boldsymbol{z}$	$m \cdot 32$ B	Final witness scalars

The verifier must confirm that $|\boldsymbol{z}| = m$, where m is recomputed from Algorithm 1 (Section 4.6).

4.6 Iterative Padding and Round Dimensions

The k-ary folding in both the IPA and eCP sub-protocols requires the vector dimension to be divisible by k at every round. When this does not hold, the vectors are padded to the next multiple of k before folding. This section specifies how the verifier *reconstructs* the round dimensions from the proof parameters (n, k, d): it needs these dimensions to perform the scalar-expansion step (Verification 4 for IPA, Verification 5 for eCP) that converts the small final witness back into n-dimensional verification scalars.

Algorithm 1: Round Length Reconstruction

Given the initial dimension n, folding factor k, and number of rounds d (read from the sub-proof header):

Algorithm 1: `reconstruct_round_lengths`(n, k, d)

$\ell_0 \leftarrow n$
for $r = 0, 1, \ldots, d-1$ **do**
  $\text{rem} \leftarrow \ell_r \bmod k$
  $\text{pad}_r \leftarrow \begin{cases} 0 & \text{if rem} = 0 \\ k - \text{rem} & \text{otherwise} \end{cases}$
  $\ell_{r+1} \leftarrow (\ell_r + \text{pad}_r)/k$
end for
$m \leftarrow \ell_d$ ▷ must match the m read from the proof
return $(\ell_0, \ell_1, \ldots, \ell_d)$

Consistency check. After running Algorithm 1, the verifier must confirm: (a) $m = \ell_d$ equals the m from the sub-proof header; (b) $m \geq 1$; (c) $\ell_{d-1} > 1$, i.e. the final folding round still reduced the dimension. Clause (c) restates check (1.L) of Section 8.1: together with $n \geq 2$ (check 1.B) it enforces $1 \leq d \leq \lceil \log_k n_{\text{circuit}} \rceil$. If any check fails, reject.

How the verifier uses the round lengths. The round-length sequence $(\ell_0, \ell_1, \ldots, \ell_d)$ is consumed during verification as follows. In Verification 4 (IPA) and Verification 5 (eCP), the verifier expands the small final witness ($\boldsymbol{a}_f$ and $\boldsymbol{b}_f$ for IPA, $\boldsymbol{z}$ for eCP) back to the original dimension n by processing the folding rounds in reverse. At each expansion step from round $r+1$ to round r, the expanded vector has $\ell_{r+1} \cdot k$ entries, but only the first ℓ_r correspond to actual (non-padded) positions. The verifier therefore truncates to ℓ_r entries after each step. After all d rounds, the result has $\ell_0 = n$ entries. These expanded scalars are then multiplied against the CRS generators $\boldsymbol{g}$ and $\boldsymbol{h}$ in the Mega-MSM (Section 8.6.2).

Worked example: $n = 10$**,** $k = 4$**.**

Round r	ℓ_r	rem	pad	ℓ_{r+1}
0	10	2	2	$(10+2)/4 = 3$
1	3	3	1	$(3+1)/4 = 1$

Result: $d = 2$, $m = 1$. The IPA sub-proof contains $2 \times (2 \cdot 4 - 2) = 12$ cross-term points and final witnesses $\boldsymbol{a}_f, \boldsymbol{b}_f \in \mathbb{Z}_p^1$.

5 Deterministic Generator Derivation (CRS)

All public generators used in MAYA are derived deterministically from the group's basepoint using standard hash functions and the FromUniformBytes map (Section 3.1). The verifier must recompute them independently and must not accept externally supplied generators.

The following quantities determine the dimensions of the vectors used throughout the protocol.

Symbol	Definition
n	Number of ciphertexts (read from the election record, Section 4.2).
n_c	Number of multiplication gates in the permutation circuit: $n_c = n-1$ (Section 8.3.1).
n_{circuit}	n rounded up to the next multiple of the folding factor k: $n_{\text{circuit}} = n + ((k - n \bmod k) \bmod k)$. This is the dimension of the committed-variable vector, the challenge vector $\boldsymbol{e}$ (Section 7), and the IPA/eCP input vectors.
N	Generator capacity: $N = n+k-1$. The Bulletproof generator vectors $\boldsymbol{g}$ and $\boldsymbol{h}$ of length N are guaranteed to contain enough generators for the padded circuit dimension.

These satisfy $n_c < n \leq n_{\text{circuit}} \leq N$. The generators $\boldsymbol{g}$ and $\boldsymbol{h}$ are indexed from 1 to N. The positions $n_{\text{circuit}} < i \leq N$ are generated but may not all be used in a given proof.

General method. The derivation follows a standard "nothing-up-my-sleeve" pattern: choose a canonical seed (the basepoint encoding), use two counter-indexed HMAC calls with a unique domain-separation label to produce $2 \times 32 = 64$ uniform bytes, and apply FromUniformBytes to obtain a group element. This is the same two-call counter-mode pattern used for the Bulletproof generators (Algorithm 2) and for the challenge vector expansion (Algorithm 3). Because FromUniformBytes produces a uniformly distributed group element, no party can know the discrete-log relation between the derived generators and the basepoint – the property required by the DLR assumption [4].

Label conventions. The string labels used throughout the CRS derivation and the Fiat–Shamir transcript (e.g. `"PedersenGens_F"`, `"GeneratorsChain"`, `"MAYA-Shuffle-v1"`) are *normative*: changing any label changes the hash output and therefore changes every derived generator or challenge scalar. A verifier must use exactly the labels specified in this document. A summary of all labels is provided in Section 6.4, while the complete reference appears in Appendix B.

5.1 Pedersen Generators

Three generators are derived from the Ristretto255 basepoint B. Let $B_0 = \mathsf{Compress}(B)$ denote the 32-byte canonical encoding (Appendix A). The notation $H(k, m)$ denotes HMAC-SHA-256 with key k and message m (Eq. 9), the same function used throughout the transcript (Section 6.2). Details of the hash-to-group map are provided in Section 6.3.

Algorithm 1 Pedersen generator derivation

$B \leftarrow$ Ristretto255 basepoint (hardcoded) ▷ ElGamal generator (g)
$B_0 \leftarrow \mathsf{Compress}(B)$ ▷ 32 bytes

$\eta_{h,1} \leftarrow H(B_0,\ \texttt{"PedersenGens\_blinding"} \,\|\, \texttt{0x01})$
$\eta_{h,2} \leftarrow H(B_0,\ \texttt{"PedersenGens\_blinding"} \,\|\, \texttt{0x02})$
$B_{\text{blinding}} \leftarrow \mathsf{FromUniformBytes}(\eta_{h,1} \| \eta_{h,2})$ ▷ Pedersen blinding base (h)

$\eta_{F,1} \leftarrow H(B_0,\ \texttt{"PedersenGens\_F"} \,\|\, \texttt{0x01})$
$\eta_{F,2} \leftarrow H(B_0,\ \texttt{"PedersenGens\_F"} \,\|\, \texttt{0x02})$
$F \leftarrow \mathsf{FromUniformBytes}(\eta_{F,1} \| \eta_{F,2})$ ▷ ElGamal public-key base (f)

The verifier must confirm that the received (g, f, h) are identical to the derived $(B, F, B_{\text{blinding}})$ (check (1.I) in Section 8.1).

5.2 Bulletproof Generators

The generator vectors $\boldsymbol{g} = (g_1, \ldots, g_N)$ and $\boldsymbol{h} = (h_1, \ldots, h_N)$ are derived using HMAC-SHA-256 in counter mode. Details of the hash-to-group map are provided in Section 6.3. For each type ("G" or "H"), a 32-byte seed is derived by a single HMAC call, and then each generator is produced by two counter-indexed HMAC calls whose outputs are concatenated and passed to FromUniformBytes. Because the counter-mode calls are independent, all N generators can be computed in parallel.

Algorithm 2 Bulletproof generator derivation (HMAC counter mode)

for type $T \in \{\texttt{0x47}, \texttt{0x48}\}$ **do** ▷ ASCII "G" and "H"
  $L \leftarrow [T,\ 0,\ 0,\ 0,\ 0]$ ▷ Single-prover index j=0 as u32 LE
  $seed \leftarrow H(\texttt{"GeneratorsChain"} \,\|\, \texttt{0x00}^{17},\ L)$ ▷ HMAC-SHA-256 (Eq. 9)
  for $i = 0, 1, \ldots, N-1$ **do**
    $\eta_{i,1} \leftarrow H\big(seed,\ \texttt{"bp\_"} \,\|\, \mathsf{b}_4(i) \,\|\, \texttt{0x01}\big)$ ▷ 8-byte message
    $\eta_{i,2} \leftarrow H\big(seed,\ \texttt{"bp\_"} \,\|\, \mathsf{b}_4(i) \,\|\, \texttt{0x02}\big)$
    g_{i+1} (or h_{i+1}) $\leftarrow \mathsf{FromUniformBytes}(\eta_{i,1} \| \eta_{i,2})$ ▷ 64 bytes → group element
  end for
end for

The prefix "bp_" is 3 bytes (0x62 0x70 0x5F); $\mathsf{b}_4(i)$ is the 4-byte little-endian encoding of index i; the final byte (0x01 or 0x02) selects the first or second 32-byte half.

The verifier generates $N = n + k - 1$ generators for each of $\boldsymbol{g}$ and $\boldsymbol{h}$. The iterative padding procedure may extend a vector of length n by up to $k - 1$ elements in order to reach the next multiple of k, so $N \geq n_{\text{circuit}}$ always holds (check (1.J) in Section 8.1).

The 32-byte seeds $seed_G$ and $seed_H$ are committed to the challenge-vector transcript (Algorithm 3, lines 7–8) to bind $\boldsymbol{e}$ to the CRS in use.

6 Hash Computation and Fiat–Shamir Transcript

This section defines the hash function used for the Fiat–Shamir transformation and the transcript state machine that turns the interactive proof into a non-interactive one. Section 6.1 identifies the single hash primitive. Section 6.2 defines the transcript hash function H. Section 6.3 specifies the transcript state machine, i.e., the three operations (Init, Append, ChallengeScalar). Section 6.4 gives the complete transcript schedule: the exact sequence of operations that a correct verifier replays to derive every Fiat–Shamir challenge. The transcript schedule is the normative reference; an implementation is correct if and only if it reproduces the schedule byte-for-byte.

6.1 Hash Primitives Used in MAYA

MAYA uses a single standard hash primitive for all operations: CRS derivation, Fiat–Shamir transcript generation, and challenge vector expansion. Concretely, the protocol uses HMAC-SHA-256 for:

- Pedersen generator derivation (§5.1),
- Bulletproof generator derivation (§5.2),
- Fiat–Shamir transcript generation (§6.3),
- and challenge vector expansion (§7).

HMAC-SHA-256 provides a deterministic keyed hash function with 32-byte output. Two counter-indexed calls produce the 64 bytes needed by FromUniformBytes. The single-primitive design minimises the dependency surface for cross-language verifier implementations.

HMAC-SHA-256 is available in all mainstream cryptographic libraries, including OpenSSL, Python's `hmac` and `hashlib`, Java's `javax.crypto.Mac`, Go's `crypto/hmac` and `crypto/sha256`, Rust's `hmac` and `sha2` crates, Node.js `crypto.createHmac`, and .NET's `HMACSHA256`. Any conforming implementation may use an equivalent cryptographic library, provided that the resulting outputs match the definitions in this specification.

6.2 The Transcript Hash Function H

The single hash function used for all transcript operations is:

$$H : \mathcal{B}^{32} \times \mathcal{B}^{*} \to \mathcal{B}^{32}, \quad (key, msg) \mapsto \text{HMAC-SHA-256}(key, msg). \tag{9}$$

The first argument (*key*) is always a 32-byte value. In every call within the transcript state machine below, this argument is the current transcript state σ. There is no secret key; the term "key" refers only to the first argument of the HMAC construction. The second argument (*msg*) has arbitrary length. The output is always exactly 32 bytes.

6.3 Transcript State Machine

The Fiat–Shamir transcript is a stateful object with a single 32-byte state σ. It supports four operations: initialization, commit (data absorption), challenge (scalar derivation), and challenge32 (byte derivation). Every Fiat–Shamir challenge in MAYA is derived through this machine.

The four operations are specified below. In each equation, H refers to the function defined in Eq. (9): the first argument is the HMAC key, the second is the HMAC message.

Init(*label*): Create a new transcript and set the initial state.

$$\sigma \leftarrow H\big(\underbrace{\texttt{0x00} \cdots \texttt{0x00}}_{\text{32 bytes}},\ \ label\big) \tag{10}$$

The key for the first HMAC call is 32 zero bytes (`0x00...00`), a public constant. The message is the label encoded as raw UTF-8 bytes (Section 4.1). For the main MAYA transcript, *label* is `"MAYA-Shuffle-v1"` (15 bytes).

Example. Init(`"MAYA-Shuffle-v1"`) computes $\sigma \leftarrow$ HMAC-SHA-256($\texttt{00...00}_{32}$, `4d4159412d536875...7631`$_{15}$), where `4d4159412d...` is the UTF-8 encoding of the label.

Append(*label*, *data*): Absorb a labeled message into the transcript.

$$\sigma \leftarrow H\big(\sigma,\ \ label \,\|\, \texttt{0x00} \,\|\, |data|_{\mathrm{LE32}} \,\|\, data\big) \tag{11}$$

The HMAC key is the current state σ. The HMAC message is the concatenation of four parts: the label (raw UTF-8, no length prefix), a single zero byte `0x00` (separator), the byte length of *data* encoded as a 4-byte little-endian unsigned integer ($|data|_{\mathrm{LE32}}$), and the data bytes. The zero byte and the length field provide domain separation: they ensure that different label/data pairs always produce different HMAC messages.

Example. Append(`"dom-sep"`, `"ShuffleProof"`), i.e., `"dom-sep"` is the *label* argument and `"ShuffleProof"` is the *data* argument, builds the HMAC message `646f6d2d736570` `00` `0c000000` `536875...6f6f66` (7-byte label, separator, length 12 = `0x0c` as LE32, 12-byte data), and computes $\sigma \leftarrow H(\sigma,$ that message$)$.

Three typed variants build the data argument from typed values before calling Append:

Operation	Definition
CommitPoint($label$, P)	Append$\big(label,\ \mathsf{Compress}(P)\big)$ *(data = 32 bytes)*
CommitScalar($label$, s)	Append$\big(label,\ \mathsf{b}(s)\big)$ *(data = 32 bytes)*
CommitU64($label$, n)	Append$\big(label,\ \mathsf{b}_8(n)\big)$ *(data = 8 bytes)*

These are the operations used in the transcript schedule (Section 6.4). Compress, $\mathsf{b}(\cdot)$, and $\mathsf{b}_8(\cdot)$ are the encoding functions defined in Section 4.1. When a small integer m is committed via CommitScalar($label$, m), the integer is first embedded into the scalar field and encoded as a 32-byte canonical little-endian scalar $\mathsf{b}(m)$: bytes 0–7 hold m in little-endian, bytes 8–31 are zero.

ChallengeScalar($label$): Derive a uniformly distributed scalar in $\mathbb{Z}_p$ from the current transcript state. This operation performs three HMAC calls and returns one scalar.

The scalar must be drawn from $\mathbb{Z}_p$ with negligible bias. Because the scalar field order is $p \approx 2^{252}$, generating a single 32-byte hash (256 bits) and reducing it modulo p yields an statistical bias of approximately $p/2^{256} \approx 2^{-4}$. To reduce this bias to a negligible margin, the operation produces 64 bytes (via two HMAC calls), interprets them as a 512-bit integer, and reduces modulo p. Because this 512-bit integer is overwhelmingly larger than p, the statistical distance from a uniform distribution is bounded by $p/2^{512} \approx 2^{-260}$, which safely exceeds standard cryptographic security margins. A third HMAC call updates the transcript state σ, binding all subsequent operations to this challenge.

$$\eta_1 \leftarrow H\big(\sigma,\ \texttt{"chal"} \,\|\, \texttt{0x00} \,\|\, label\big) \tag{12}$$

$$\eta_2 \leftarrow H\big(\sigma,\ \texttt{"chal\_ext"} \,\|\, \texttt{0x00} \,\|\, \eta_1\big) \tag{13}$$

$$s \leftarrow \mathrm{int}_{\mathrm{LE}}(\eta_1 \| \eta_2) \bmod p \tag{14}$$

$$\sigma \leftarrow H\big(\sigma,\ \texttt{"chal\_update"} \,\|\, \texttt{0x00} \,\|\, \eta_1\big) \tag{15}$$

where:

- $\eta_1, \eta_2 \in \mathcal{B}^{32}$ are intermediate hash outputs (they are internal to this operation and are never exposed except through s).
- $\mathrm{int}_{\mathrm{LE}}(\eta_1 \| \eta_2)$ interprets the 64-byte concatenation as a 512-bit *little-endian* unsigned integer (byte 0 is the least significant).
- The strings `"chal"` (4 bytes), `"chal_ext"` (8 bytes), and `"chal_update"` (11 bytes) are hardcoded protocol constants, they are the same for every challenge derivation in every transcript. The label argument (which varies per call) distinguishes different challenges.
- All three HMAC calls use the *same* state σ (the state before this operation) as the key. Only Eq. (15) updates σ.

The operation returns the scalar s and leaves the transcript in the updated state.

Example. ChallengeScalar(`"y"`): Eq. (12) computes $\eta_1 = H(\sigma,\ \texttt{6368616c 00 79})$ (where `6368616c` = `"chal"` and `79` = `"y"`). Eq. (13) computes $\eta_2 = H(\sigma,\ \texttt{6368616c5f657874 00}\,\eta_1)$. The 64 bytes $\eta_1 \| \eta_2$ are read as a little-endian integer and reduced mod p to give the challenge scalar y. Finally, Eq. (15) updates $\sigma \leftarrow H(\sigma,\ \texttt{6368616c5f757064617465 00}\,\eta_1)$.

ChallengeBytes32($label$): Derive 32 uniform bytes (rather than a scalar) from the current transcript state. This variant is used to extract the PRG seed for the challenge-vector derivation (Algorithm 3, Section 7). It performs the first and last HMAC calls of ChallengeScalar and skips the extension call:

$$\eta_1 \leftarrow H\big(\sigma,\ \texttt{"chal"} \,\|\, \texttt{0x00} \,\|\, label\big) \tag{16}$$

$$\sigma \leftarrow H\big(\sigma,\ \texttt{"chal\_update"} \,\|\, \texttt{0x00} \,\|\, \eta_1\big) \tag{17}$$

The operation returns the 32-byte value η_1 and leaves the transcript in the updated state.

6.4 Complete Transcript Schedule

The preceding subsections defined the hash function H (Section 6.2) and the three transcript operations Init, Append, and ChallengeScalar (Section 6.3). This subsection specifies the *exact sequence* of those operations that a conforming verifier must replay on the main transcript. A verifier implementation is correct if and only if it executes the operations below in the order shown, with the exact labels and values listed, and obtains the same challenge scalars.

The table uses two symbols: $\triangleright$ denotes a commit operation (Append, CommitPoint, CommitScalar, or CommitU64), and $\Leftarrow$ denotes a challenge derivation (ChallengeScalar). The right arrow $\Rightarrow$ shows the variable that receives the derived scalar.

#	Op	Label and value
0	Init	`"MAYA-Shuffle-v1"` (15 bytes)
1	$\triangleright$	Append(`"dom-sep"`, `"ShuffleProof"`) (12 bytes)
2	$\triangleright$	CommitScalar(`"circuit_n"`, Scalar(n_{circuit})) (32 bytes)[1]
3	$\triangleright$	Append(`"dom-sep"`, `"r1cs v1"`) (7 bytes)
4	$\triangleright$	CommitPoint(`"V"`, V) (32 bytes)
5	$\triangleright$	CommitU64(`"m"`, n_{circuit}) (8 bytes)
6	$\Leftarrow$	ChallengeScalar(`"shuffle challenge"`) $\Rightarrow$ u[2]
7	$\triangleright$	CommitPoint(`"A_I"`, A_I)
8	$\triangleright$	CommitPoint(`"A_O"`, A_O) [3]
9	$\triangleright$	CommitPoint(`"S"`, S)
10	$\Leftarrow$	ChallengeScalar(`"y"`) $\Rightarrow$ y
11	$\Leftarrow$	ChallengeScalar(`"z"`) $\Rightarrow$ z
12	$\triangleright$	CommitPoint(`"T_1"`, T_1)
13	$\triangleright$	CommitPoint(`"T_3"`, T_3)
14	$\triangleright$	CommitPoint(`"T_4"`, T_4)
15	$\triangleright$	CommitPoint(`"T_5"`, T_5)
16	$\triangleright$	CommitPoint(`"T_6"`, T_6)
17	$\triangleright$	CommitPoint(`"T_2"`, T_2)[4]
18	$\Leftarrow$	ChallengeScalar(`"x"`) $\Rightarrow$ x
19	$\triangleright$	CommitScalar(`"t_x"`, t_x)
20	$\triangleright$	CommitScalar(`"t_x_blinding"`, τ_x)
21	$\triangleright$	CommitScalar(`"e_blinding"`, μ)
22	$\triangleright$	CommitPoint(`"S_prime"`, S')
23	$\triangleright$	CommitPoint(`"T_1_prime"`, T_1')
24	$\triangleright$	CommitPoint(`"S1_prime"`, S_1')
25	$\triangleright$	CommitPoint(`"S2_prime"`, S_2')
26	$\Leftarrow$	ChallengeScalar(`"x_prime"`) $\Rightarrow$ x'
27	$\triangleright$	CommitScalar(`"tc_x"`, $t_{c,x}$)
28	$\triangleright$	CommitScalar(`"tc_x_blinding"`, $\tau_{c,x}$)
29	$\triangleright$	CommitScalar(`"ec_blinding"`, μ_c)
30	$\triangleright$	CommitScalar(`"r_blinding"`, ρ_c)
31	$\triangleright$	CommitScalar(`"t_cross"`, t_{cross})
32	$\Leftarrow$	ChallengeScalar(`"x_ipp"`) $\Rightarrow$ x_{ipp}
33	$\Leftarrow$	ChallengeScalar(`"w_agg"`) $\Rightarrow$ w_{agg}
— *IPA sub-transcript replay (Algorithm 4, Phase A, Section 9)* —		
34	$\Leftarrow$	ChallengeScalar(`"chall_batched_ecp"`) $\Rightarrow$ α_{ecp}
— *eCP sub-transcript replay (Algorithm 5, Phase A, Section 10)* —		

[1] The *padded circuit dimension* n_{circuit} (Section 5) encoded as a 32-byte scalar. When n is a multiple of the folding factor k, the value equals Scalar(n).

[2] The shuffle challenge u is derived *before* the proof header points (A_I, A_O, S), not after.

[3] capital letter O for Output, not digit zero

[4] T_2 (the bridging commitment) is committed last among $T_1, \ldots, T_6$.

35	$\Leftarrow$	ChallengeScalar("batching_r") $\Rightarrow$ r	

The values $V, A_I, A_O, S, T_1, \ldots, T_6, S', T'_1, S'_1, S'_2$ are points read from the proof (Section 4.3). The values $t_x, \tau_x, \mu, t_{c,x}, \tau_{c,x}, \mu_c, \rho_c, t_{\text{cross}}$ are scalars read from the proof. The variable n_{circuit} is the padded circuit dimension (Section 5). The sequence above is a single, strictly ordered stream of operations on *one* transcript object $\mathcal{T}$:

1. After deriving w_{agg} at step #33, the IPA sub-transcript begins *immediately*: the next operation on $\mathcal{T}$ is `Append("protocol-name", "ipa_fold")` of Algorithm 4.
2. α_{ecp} (step #34) is derived on $\mathcal{T}$ *after* the last IPA round challenge c_{d-1} and *before* the eCP sub-transcript's `Append("protocol-name", "ecp_fold")`.
3. r (step #35) is derived on $\mathcal{T}$ *after* the last eCP round challenge c^e_{d-1}. It is the final transcript operation.

All labels in the table above are *normative*: changing a single byte in any label changes the HMAC output and therefore changes every subsequent challenge. Deriving any of these challenges at a different position, or on a fresh transcript, changes every subsequent challenge and causes the Mega-MSM to reject.

7 Challenge Vector Derivation

The public challenge vector $\boldsymbol{e} = (e_1, \ldots, e_{n_{\text{circuit}}}) \in \mathbb{Z}_p^{n_{\text{circuit}}}$ is the random fingerprint of positions that reduces the shuffle check to the polynomial identity of Eq. (5) (Section 2.1). For soundness, $\boldsymbol{e}$ must be determined only *after* the shuffled ciphertexts $\boldsymbol{C}'$ are fixed; otherwise the prover could choose the permutation adaptively as a function of $\boldsymbol{e}$ [1]. In the interactive protocol, the verifier samples $\boldsymbol{e} \leftarrow \mathbb{Z}_p^n$ at random after receiving $\boldsymbol{C}'$. In the non-interactive setting specified here, $\boldsymbol{e}$ is derived via the Fiat–Shamir transformation: a seed z_e is computed by hashing the entire public statement (public key, CRS identifiers, and both ciphertext lists), and the vector is expanded as $\boldsymbol{e} = \text{PRG}(z_e)$ using a pseudorandom generator. Accordingly, the derivation has two stages:

- *Stage 1 (seed extraction).* All public inputs are absorbed into a dedicated transcript, which is separate from the main transcript of Section 6.4, and a 32-byte seed z_e is extracted from its final state. Binding the seed to the complete public statement is what prevents adaptive choices: any change to a single ciphertext byte changes the seed and therefore the whole vector $\boldsymbol{e}$.
- *Stage 2 (counter-mode expansion).* Each scalar e_i is expanded from the seed by two counter-indexed HMAC calls, whose 64 concatenated output bytes are reduced modulo p—the same bias-reduction technique as ChallengeScalar (Section 6.3).

In the algorithm 3, g, f, h are the ElGamal and Pedersen generators read from the election record (Algorithm 1 and Section 4.2); seed_G, seed_H are the 32-byte HMAC seeds from Algorithm 2 (Section 5.2); $C_{j,1}, C_{j,2}$ and $C'_{j,1}, C'_{j,2}$ are the input and output ciphertext components; and n_{circuit} is the padded circuit dimension (Section 5).

Stage 1 consists of the transcript operations defined in Section 6.3, applied to the dedicated transcript $\mathcal{T}_e$. The seed extraction in the last step is the ChallengeBytes32 operation (Eqs. 16–17). In Stage 2, lines 15–16 are direct calls to the bare hash function H of Eq. (9), with the fixed seed z_e as the HMAC key in every call and a 7-byte counter-indexed message. The calls for different indices are independent and may be computed in any order or in parallel.

Example (Stage 2, index $i = 5$). The message of the first call is the 7-byte array

$$\underbrace{\texttt{0x65 0x5F}}_{\texttt{"e\_"}} \,\|\, \underbrace{\texttt{0x05 0x00 0x00 0x00}}_{\mathsf{b}_4(5)} \,\|\, \underbrace{\texttt{0x01}}_{\text{counter byte}} ,$$

Algorithm 3 Challenge vector $\boldsymbol{e}$ derivation

Stage 1: seed extraction

1: $\mathcal{T}_e \leftarrow \mathsf{Init}(\texttt{"FiatShamir-e"})$ ▷ separate transcript; state machine of §6.3
2: $\mathcal{T}_e.\mathsf{Append}(\texttt{"dom-sep"},\ \texttt{"challenge-e-derivation"})$
3: $\mathcal{T}_e.\mathsf{CommitPoint}(\texttt{"pk\_g"},\ g)$
4: $\mathcal{T}_e.\mathsf{CommitPoint}(\texttt{"pk\_f"},\ f)$
5: $\mathcal{T}_e.\mathsf{CommitPoint}(\texttt{"ck\_h"},\ h)$
6: $\mathcal{T}_e.\mathsf{CommitU64}(\texttt{"ck\_n"},\ N)$
7: $\mathcal{T}_e.\mathsf{Append}(\texttt{"ck\_g\_seed"},\ \mathit{seed}_G)$ ▷ 32-byte CRS seed (§5.2)
8: $\mathcal{T}_e.\mathsf{Append}(\texttt{"ck\_h\_seed"},\ \mathit{seed}_H)$
9: $\mathcal{T}_e.\mathsf{Append}(\texttt{"c1"},\ \mathsf{Compress}(C_{1,1})\|\cdots\|\mathsf{Compress}(C_{n,1}))$ ▷ one call, $32n$ bytes
10: $\mathcal{T}_e.\mathsf{Append}(\texttt{"c2"},\ \mathsf{Compress}(C_{1,2})\|\cdots\|\mathsf{Compress}(C_{n,2}))$
11: $\mathcal{T}_e.\mathsf{Append}(\texttt{"c1p"},\ \mathsf{Compress}(C'_{1,1})\|\cdots\|\mathsf{Compress}(C'_{n,1}))$
12: $\mathcal{T}_e.\mathsf{Append}(\texttt{"c2p"},\ \mathsf{Compress}(C'_{1,2})\|\cdots\|\mathsf{Compress}(C'_{n,2}))$
13: $z_e \leftarrow \mathcal{T}_e.\mathsf{ChallengeBytes32}(\texttt{"e\_seed"})$ ▷ 32-byte seed (Eqs. 16–17); $\mathcal{T}_e$ is not used again

Stage 2: counter-mode expansion

14: **for** $i = 0, 1, \ldots, n-1$ **do**
15: $\quad\eta_{i,1} \leftarrow H\big(z_e,\ \texttt{"e\_"} \,\|\, \mathsf{b}_4(i) \,\|\, \texttt{0x01}\big)$ ▷ direct call to H, Eq. 9
16: $\quad\eta_{i,2} \leftarrow H\big(z_e,\ \texttt{"e\_"} \,\|\, \mathsf{b}_4(i) \,\|\, \texttt{0x02}\big)$
17: $\quad e_{i+1} \leftarrow \mathrm{int}_{\mathrm{LE}}(\eta_{i,1}\|\eta_{i,2}) \bmod p$ ▷ as in Eq. 14
18: **end for**
19: $e_{n+1}, \ldots, e_{n_{\mathrm{circuit}}} \leftarrow 0$ ▷ zero-pad to circuit dimension (§5)

so $\eta_{5,1} = \text{HMAC-SHA-256}(z_e,\ \texttt{0x655F050000000001})$, and the second call uses the same message with final byte `0x02`. The 64 bytes $\eta_{5,1}\|\eta_{5,2}$ are interpreted as a little-endian 512-bit integer and reduced modulo p to give the scalar e_6 (the entry for position 6, since entries are indexed from 1 while the counter starts at 0).

Lines 7–8 commit the 32-byte HMAC seeds seed_G and seed_H, rather than the generators themselves. This is sufficient because each seed fully determines the corresponding generator vector: for any index i, the generator g_{i+1} is a deterministic function of seed_G and i alone (Algorithm 2). Committing N (line 6) and the two seeds (lines 7–8) therefore binds the complete generator set: any change to the seed, the derivation labels, or the capacity would change the transcript state and therefore the challenge vector $\boldsymbol{e}$. Recall also that the verifier never accepts externally supplied generators: it recomputes the full CRS itself (Section 5). The seed commitments here serve to bind the challenge vector to the CRS in use, not to authenticate the generators.

8 Verification Steps

This section collects the steps to verify a MAYA proof. The verification consists of six numbered verifications, executed in order. Each verification is presented as a framed block containing two parts, separated by a horizontal rule:

1. *Required computations* (numbered x.1, x.2, ...): intermediate values the verifier must compute.
2. *Required checks* (lettered x.A, x.B, ...): conditions the verifier must confirm. If any check fails, the verifier outputs Reject immediately.

Each verification block is accompanied by a variable table that lists every symbol used and the section in which it is defined. The proof is Accept only if every check of every verification succeeds.

The arithmetic operations required, i.e., scalar field operations, scalar multiplication, multi-scalar multipli- cation, point decompression, and the hash primitives, are described in Section 3. The entire procedure is deterministic: given the same input bytes, any conforming implementation produces the same accept/reject decision.

8.1 Input Validation

Verification 1 (Input Validation)

A MAYA verifier must confirm that the input bytes parse into well-formed objects of the correct lengths, that the group parameters match the Ristretto255 specification, and that the derived CRS is consistent.

Required computations:

(1.1) Parse the election record (Section 4.2) and the proof (Sections 4.3–4.5), reading n, the ciphertext vectors, the generators (g, f, h), the commitment V, the proof header, and the sub-proof headers (k, d, m).
(1.2) Compute the dimensions (Section 5): the padded circuit dimension $n_{\text{circuit}} = n + ((k - n \bmod k) \bmod k)$, the number of multiplication gates $n_c = n - 1$, and the generator capacity $N = n + k - 1$. Run Algorithm 1 (Section 4.6) on $(n_{\text{circuit}}, k, d)$ to obtain the round lengths $(\ell_0, \ldots, \ell_d)$ and the expected base-case size ℓ_d.
(1.3) Derive the CRS generators: B, B_{blinding}, F (Section 5.1) and $\boldsymbol{g}, \boldsymbol{h}$ of capacity N (Section 5.2).
(1.4) Derive the challenge vector $\boldsymbol{e}$ (Section 7).

Required checks:

(1.A) The group is Ristretto255 with the basepoint encoding and scalar field order p given in Appendix A. (These values may be hardcoded.)
(1.B) $n \geq 2$.
(1.C) The folding factor satisfies $k \in \{2, 3, 4, 5, 6, 8, 16\}$. Both the IPA and eCP sub-proof headers must carry the same folding factor k and the same number of rounds d; if they disagree in either value, reject.
(1.D) All declared lengths are consistent: each ciphertext vector $(\boldsymbol{c}_1, \boldsymbol{c}_2, \boldsymbol{c}'_1, \boldsymbol{c}'_2)$ contains exactly n elements ($32n$ bytes each), and the proof byte length equals $688 + \ell_{\text{IPA}} + \ell_{\text{eCP}}$ (Section 4.3).

> **Implementation note.** Before parsing, verify that the input file contains at least $8 + 128n + 96$ bytes and that the proof bundle contains at least $8 + \ell + 32 + 8 + 32n + 8 + 32n$ bytes. In languages with implicit array bounds (e.g., Python), reading past the end of a byte array may silently produce truncated slices; in languages with explicit memory management (C, C++), it is undefined behavior.

(1.E) The sub-proof body has the exact expected size: the IPA sub-proof contains $d(2k-2)$ cross-term points and final witnesses with $|\boldsymbol{a}_f| = |\boldsymbol{b}_f| = m = \ell_d$; the eCP sub-proof contains $d(2k-2)$ cross-term point pairs and a final witness with $|\boldsymbol{z}| = m = \ell_d$.
(1.F) Every scalar in the proof (the 8 header scalars, $\boldsymbol{a}_f$, $\boldsymbol{b}_f$, $\boldsymbol{z}$) is canonical: the 32-byte little-endian integer satisfies $0 \leq s < p$ (Section 4.1).
(1.G) Every group element decompresses successfully via Decompress (RFC 9496, §4.3.1): the generators (g, f, h), all $4n$ ciphertext components, V, the 13 proof header points, and all sub-proof cross-term points (Sections 4.4 and 4.5).
(1.H) The generators f and h are not the identity element.
(1.I) The derived CRS generators match the election-record values as $B = g$, $B_{\text{blinding}} = h$, $F = f$ (Section 5.1).
(1.J) The generator capacity suffices: $N \geq n_{\text{circuit}}$.

Verification 1 (Input Validation)

(1.K) In each sub-proof header (§4.4, §4.5), the three parameters k, d, m occupy 32-byte slots with the u64 value in bytes 0–7; bytes 8–31 of each slot must be zero. Reject otherwise. (This makes the encoding injective; without it, two distinct byte strings decode to the same proof.)
(1.L) The number of rounds satisfies $1 \leq d \leq \lceil \log_k n_{\text{circuit}} \rceil$ (equivalently, $\ell_{d-1} > 1$ in the output of Algorithm 1). This rejects degenerate proofs that append extra folding rounds of dimension $1 \to 1$ and bounds the verifier's work as a function of n alone.
(1.M) The verifier must ensure that all length, offset, and allocation computations can be represented without integer overflow in the implementation language. Implementations may impose deployment-specific limits on n.
(1.N) The output-side counters equal the input count: $n' = n'' = n$ (§4.2). Reject otherwise. The input file and the proof bundle must each have *exactly* the length implied by their declared counters; trailing bytes after the last field cause rejection.

The sub-proof byte lengths are:

$$\ell_{\mathsf{IPA}} = 3 \cdot 32 + d(2k{-}2) \cdot 32 + 2m \cdot 32 \quad (\mathsf{IPA}\text{: 3 header scalars, cross-terms, final witnesses}),$$
$$\ell_{\mathsf{eCP}} = 3 \cdot 32 + d(2k{-}2) \cdot 2 \cdot 32 + m \cdot 32 \quad (\mathsf{eCP}\text{: 3 header scalars, cross-term pairs, final witness}),$$

where k, d, and $m = \ell_d$ are read from the sub-proof headers (Sections 4.4 and 4.5). The factor of 2 in the eCP cross-terms accounts for the point pairs $[A^{(0)}, A^{(1)}]$.

Variable	Description	Section
p	Scalar field order of Ristretto255	App. A
n	Number of ciphertexts	§4.2
k, d, m	Folding factor, rounds, base size	§4.4
n_c	Number of gates $(n{-}1)$	§8.3.1
n_{circuit}	Padded circuit dimension	§5
N	Generator capacity $(n{+}k-1)$	§5
g, f, h	ElGamal generator, public-key base, blinding base	§4.2, §5.1
$B, B_{\text{blinding}}, F$	Derived CRS generators $(= g, h, f)$	§5.1, §5.1
V	Vector commitment	§4.2
$\boldsymbol{g}, \boldsymbol{h}$	Bulletproof generators	§5.2
$\boldsymbol{e}$	Challenge vector	§7

8.2 Challenge Derivation

Verification 2 (Transcript Replay and Challenge Derivation)

The verifier must re-derive every Fiat–Shamir challenge by replaying the main transcript. The normative sequence of operations is given by the Complete Transcript Schedule (Section 6.4).

Required computations:

(2.1) Initialize the main transcript: $\mathcal{T} \leftarrow \mathsf{Init}(\texttt{"MAYA-Shuffle-v1"})$ (schedule step #0).
(2.2) Execute schedule steps #1–#33 in order exactly as listed in Section 6.4. The challenges $c_0, \ldots, c_{d-1}$ (IPA rounds), α_{ecp}, $c^e_0, \ldots, c^e_{d_e-1}$ (eCP rounds), and r are derived later on the *same* transcript, inside Verifications 4 and 5, at the schedule positions shown in the table below.

Required checks:

(2.A) Each challenge is derived through the ChallengeScalar operation (Eqs. 12–15) at its exact schedule position with the exact label shown in Section 6.4.
(2.B) Every derived challenge scalar is non-zero. If any challenge equals 0 mod p, Reject.

Challenge	Label	Schedule #	First used	Section
u	`"shuffle challenge"`	6	Verif. 3	§6.4
y	`"y"`	10	Verif. 5	§6.4
z	`"z"`	11	Verif. 3	§6.4
x	`"x"`	18	Verif. 5	§6.4
x'	`"x_prime"`	26	Verif. 6	§6.4
x_{ipp}	`"x_ipp"`	32	Verif. 5	§6.4
w_{agg}	`"w_agg"`	33	Verif. 5	§6.4
c_j	`"challenge_separator"`	IPA	Verif. 4	§B.3
α_{ecp}	`"chall_batched_ecp"`	post-IPA	Verif. 5	§6.4
c^e_j	`"challenge_separator"`	eCP	Verif. 5	§B.4
r	`"batching_r"`	last	Verif. 5	§6.4

8.3 Permutation Circuit

8.3.1 The Circuit Construction

The permutation check of Eq. (5) is encoded as an arithmetic circuit with $n_c = n - 1$ multiplication gates. The circuit uses n_{circuit} committed variables $e'_0, \ldots, e'_{n_{\text{circuit}}-1}$ and the public challenge vector $e_0, \ldots, e_{n_{\text{circuit}}-1}$ (where $e_i = 0$ for $i \geq n$). The shuffle challenge u is derived from the transcript in step (3.1) of Verification 3 (Section 8.3, schedule step #6 of Section 6.4).

Gate allocation. The circuit creates $n_c = n-1$ multiplication gates, indexed $j = 0, 1, \ldots, n-2$, each satisfying the Hadamard relation $a_{L,j} \cdot a_{R,j} = a_{O,j}$.

Constraint generation. The circuit produces $Q = 2(n-1)+(n_{\text{circuit}}-n)+1$ linear constraints in the following order. Each multiplication gate produces two constraints (left then right), and each direct constraint produces one.

In the constraint generation steps below, each linear relation is accompanied by a list of *Terms*. This list encodes the equation into the standard R1CS form $\sum(\text{variable} \times \text{coefficient}) = 0$. For example, the relation $A = B - c$ is rearranged to $0 = 1 \cdot B - 1 \cdot A - c \cdot \mathbf{1}$, which is denoted by the terms $(B, +1)$, $(A, -1)$, and $(\mathbf{1}, -c)$, where $\mathbf{1}$ represents the constant term.

Part A: Multiplication gates ($2(n-1)$ constraints).

For each gate $j = 0, 1, \ldots, n-2$:

1. $q = 2j + 1$ (left wire):
   - If $j = 0$: $a_{L,0} = e'_0 - u$. Terms: $(a_{L,0}, -1)$, $(e'_0, +1)$, $(\mathbf{1}, -u)$.
   - If $j \geq 1$: $a_{L,j} = a_{O,j-1}$. Terms: $(a_{L,j}, -1)$, $(a_{O,j-1}, +1)$.
2. $q = 2j + 2$ (right wire): $a_{R,j} = e'_{j+1} - u$. Terms: $(a_{R,j}, -1)$, $(e'_{j+1}, +1)$, $(\mathbf{1}, -u)$.

Part B: Padding constraints ($n_{\text{circuit}} - n$ constraints, if $n_{\text{circuit}} > n$).

Because the iterative folding mechanism requires the circuit dimension to be a multiple of the folding factor k, the variable vector $\mathbf{e}'$ may be padded with dummy variables up to dimension n_{circuit}. To prevent prover manipulation, these constraints explicitly force all padded variables to be zero.

For each padded index $i = n, n+1, \ldots, n_{\text{circuit}} - 1$:

- $q = 2(n-1) + 1 + (i - n)$: $e'_i = 0$. Terms: $(e'_i, +1)$.

During padding, the running linear combination is scaled by $(-u)$ per padded position (no new gate is created).

Part C: Output constraint (1 constraint).

This final constraint enforces the polynomial equality check from Phase 1. The wire $a_{O,n-2}$ holds the prover's running product of the unpadded variables. This product is scaled by the padding correction factor $(-u)^{n_{\text{circuit}}-n}$ (since $(0 - u) = -u$ for all dummy variables) and is constrained to equal the verifier's target evaluation y_{LHS}.

Constraint $q = Q$:

$$(-u)^{n_{\text{circuit}}-n} \cdot a_{O,n-2} = y_{\text{LHS}}$$

where

$$y_{\text{LHS}} = \prod_{i=0}^{n-1} (e_i - u) \cdot (-u)^{n_{\text{circuit}}-n}$$

Terms: $(a_{O,n-2}, (-u)^{n_{\text{circuit}}-n})$, $(\mathbf{1}, -y_{\text{LHS}})$.

Flattened weight vectors. The verifier flattens the constraints by accumulating each constraint q with the power z^q. The sign convention is such that, for committed-variable and constant terms, the sign is *negated*, so the resulting scalar equation reads $\boldsymbol{w}_L \cdot \boldsymbol{a}_L + \boldsymbol{w}_R \cdot \boldsymbol{a}_R + \boldsymbol{w}_O \cdot \boldsymbol{a}_O = \boldsymbol{w}_V \cdot \boldsymbol{e}' + w_c$.

For the case $n_{\text{circuit}} = n$ (no padding), the closed-form expressions are:

$$w_L[j] = -z^{2j+1} \qquad \text{for } j = 0, 1, \ldots, n-2 \tag{18}$$

$$w_R[j] = -z^{2j+2} \qquad \text{for } j = 0, 1, \ldots, n-2 \tag{19}$$

$$w_O[j] = z^{2j+3} \qquad \text{for } j = 0, 1, \ldots, n-2 \tag{20}$$

$$w_V[0] = -z$$
$$w_V[i] = -z^{2i} \qquad \text{for } i = 1, 2, \ldots, n-1, \tag{21}$$

Implementation note. The committed-variable weight $w_V[i]$ has three distinct formulas depending on the index range: $w_V[0] = -z$; $w_V[i] = -z^{2i}$ for $1 \leq i < n$; $w_V[i] = -z^{n-1+i}$ for $n \leq i < n_{\text{circuit}}$ (padding positions, only when $n_{\text{circuit}} > n$). An implementer who applies $-z^{2i}$ uniformly including $i = 0$ computes $w_V[0] = -1$ instead of the correct $-z$.

and

$$w_c = u \cdot \left(z + z^2 + z^4 + z^6 + \cdots + z^{2(n-1)}\right) + y_{\text{LHS}} \cdot z^{2n-1} \tag{22}$$

where $y_{\text{LHS}} = \prod_{i=0}^{n-1}(e_i - u)$. The wire-weight vectors $\boldsymbol{w}_L$, $\boldsymbol{w}_R$, $\boldsymbol{w}_O$ have length $n_c = n - 1$ (the number of multiplication gates). The committed-variable weight vector $\boldsymbol{w}_V$ has length n_{circuit} (the padded circuit dimension). For positions $i \geq n_c$, the wire weights are implicitly zero: $w_{L,i} = w_{R,i} = w_{O,i} = 0$. This zero-extension is used in step (4.4e) but the vectors themselves are stored with only n_c entries.

When $n_{\text{circuit}} > n$ (padded committed variables), the formulas extend as follows. Let $\rho = n_{\text{circuit}} - n$ denote the padding count. The equations must be updated to account for the padding constraints (Part B) and the shifted Output constraint (Part C). The assignments are:

$$w_V[i] \mathrel{+}= -z^{n-1+i} \qquad \text{for } i = n, n+1, \ldots, n_{\text{circuit}} - 1 \tag{23}$$

$$w_O[n-2] \mathrel{+}= (-u)^\rho \cdot z^Q \qquad \text{(replaces the term } z^{2n-1} \text{ in Eq. 20)} \tag{24}$$

$$w_c = u \cdot \left(z + z^2 + z^4 + \cdots + z^{2(n-1)}\right) + y_{\text{LHS}} \cdot z^Q \tag{25}$$

where $Q = 2(n-1) + \rho + 1$ and $y_{\text{LHS}} = \prod_{i=0}^{n-1}(e_i - u) \cdot (-u)^\rho$.

8.3.2 Example: $n = 4$, $n_{\text{circuit}} = 4$

With $n = 4$: $n_c = 3$ gates, $Q = 7$ constraints, $y_{\text{LHS}} = (e_0 - u)(e_1 - u)(e_2 - u)(e_3 - u)$.

Constraint schedule.

q	Constraint	Linear combination
1	Gate 0, left	$(e'_0, +1) + (\mathbf{1}, -u) + (a_{L,0}, -1)$
2	Gate 0, right	$(e'_1, +1) + (\mathbf{1}, -u) + (a_{R,0}, -1)$
3	Gate 1, left	$(a_{O,0}, +1) + (a_{L,1}, -1)$
4	Gate 1, right	$(e'_2, +1) + (\mathbf{1}, -u) + (a_{R,1}, -1)$
5	Gate 2, left	$(a_{O,1}, +1) + (a_{L,2}, -1)$
6	Gate 2, right	$(e'_3, +1) + (\mathbf{1}, -u) + (a_{R,2}, -1)$
7	Output	$(a_{O,2}, +1) + (\mathbf{1}, -y_{\text{LHS}})$

Flattened weights.

$$\begin{aligned}
\boldsymbol{w}_L &= (-z,\ -z^3,\ -z^5) \\
\boldsymbol{w}_R &= (-z^2,\ -z^4,\ -z^6) \\
\boldsymbol{w}_O &= (z^3,\ z^5,\ z^7) \\
\boldsymbol{w}_V &= (-z,\ -z^2,\ -z^4,\ -z^6) \\
w_c &= u(z + z^2 + z^4 + z^6) + y_{\text{LHS}} \cdot z^7
\end{aligned}$$

Sanity check. One can verify constraint $q = 1$ by checking the contribution to the equation $\boldsymbol{w}_L \cdot \boldsymbol{a}_L + \boldsymbol{w}_R \cdot \boldsymbol{a}_R + \boldsymbol{w}_O \cdot \boldsymbol{a}_O = \boldsymbol{w}_V \cdot \boldsymbol{e}' + w_c$: the z^1 terms give $-z \cdot a_{L,0}$ on the left and $-z \cdot e'_0 + uz$ on the right, encoding $a_{L,0} = e'_0 - u$. The z^3 terms give $-z^3 a_{L,1} + z^3 a_{O,0}$, encoding the chaining $a_{L,1} = a_{O,0}$. The z^7 terms give $z^7 a_{O,2}$ on the left and $y_{\text{LHS}} \cdot z^7$ on the right, encoding the output constraint $a_{O,2} = y_{\text{LHS}}$.

Implementation note. If the flattened weights do not match the test vectors in §11:
1. Check that $w_V[0] = -z$ (not $-z^0 = -1$ and not $-z^2$).
2. Check that w_L and w_R have length $n_c = n - 1$, not n or n_{circuit}.
3. Check the sign: $w_L[j]$ and $w_R[j]$ are *negative* (the leading minus sign comes from the Bulletproofs flattening convention); $w_O[j]$ is positive.
4. Check that the z-exponents follow the interleaved pattern $q = 1, 2, 3, 4, 5, 6, \ldots$ (left, right, left, right, ...), not a grouped pattern (all left, then all right).

8.3.3 Constraint Flattening Verification

Verification 3 (Constraint Flattening)

The verifier must build the linear constraints of the Phase-1 permutation circuit (Section 8.3.1) and compress them into weight vectors using powers of z.

Required computations:

(3.1) The shuffle challenge $u \leftarrow \mathcal{T}.\mathsf{ChallengeScalar}(\texttt{"shuffle challenge"})$ at schedule step #6 (Section 6.4).
(3.2) The flattened weight vectors $\boldsymbol{w}_L, \boldsymbol{w}_R, \boldsymbol{w}_O \in \mathbb{Z}_p^{n_c}$, $\boldsymbol{w}_V \in \mathbb{Z}_p^{n_{\text{circuit}}}$, and the constant $w_c \in \mathbb{Z}_p$, using the closed-form expressions of Eqs. (18)–(22) (or Eqs. (23)–(25) when $n_{\text{circuit}} > n$).

Required checks:

(3.A) The z-power exponents follow the *interleaved* constraint ordering. See the constraint schedule table in Section 8.3.1 for the full ordering.
(3.B) The signs of $\boldsymbol{w}_V$ and w_c include the negation from the Bulletproofs flattening convention (Section 8.3.1).
(3.C) When $n_{\text{circuit}} > n$, the output weight $w_O[n_c - 1]$ absorbs the scaling factor $(-u)^{n_{\text{circuit}} - n}$ per Eq. (24).

The correctness of (3.A)–(3.C) can be validated against the flattened-weight test vectors for $n = 4$ (Section 11).

Variable	Description	Section
u	Shuffle challenge	§6.4, #6
z	Flattening challenge	§6.4, #11
$\boldsymbol{w}_L, \boldsymbol{w}_R, \boldsymbol{w}_O$	Wire weight vectors	§8.3.1
$\boldsymbol{w}_V$	Committed-variable weights	§8.3.1
w_c	Constant term	§8.3.1

8.4 IPA Sub-Protocol

This verification replays the IPA sub-protocol on the main transcript, expands the folded witness into per-generator scalars, and then combines those scalars with the circuit's weight vectors to produce the final generator coefficients for the Mega-MSM.

The IPA sub-protocol is defined in Section 9; it is the same inner-product argument used in Bulletproofs [5] extended to k-ary folding [1]. The sub-protocol appends to the same main transcript $\mathcal{T}$ that Verification 2 initialized in Section 8.2, so the challenges it derives are bound to everything committed so far.

Verification 4 (IPA and Phase-1 Scalar Assembly)

Required computations:

(4.1) Append the IPA sub-transcript header to $\mathcal{T}$ (continues the main transcript from schedule step #33, Section 6.4).
These are the same operations the prover executed; the verifier must replay them in the same order so that the transcript states agree:

a) Append("protocol-name", "ipa_fold"): identifies the sub-protocol.
b) Append("n", $\mathsf{b}_8(n_{\text{circuit}})$): commits the padded dimension n_{circuit} (the same value committed at schedule step #5, Section 6.4).
c) Append("k", $\mathsf{b}_8(k)$): commits the folding factor.

(4.2) For each IPA folding rounds (for $j = 0, 1, \ldots, d-1$), commit the $2k-2$ cross-term points from the proof and derive one round challenge, following the per-round transcript operations of Algorithm 4, Phase A (Section 9). These cross-terms are the prover's "hints" that allow the dimension to shrink by a factor of k at each round while preserving the verification equation.
Per round j:

a) For each cross-term index $\ell = 0, \ldots, 2k-3$:
   - Append("U_round", $\mathsf{b}_8(j)$)
   - Append("U_index", $\mathsf{b}_8(\ell)$)
   - CommitPoint("U_point", $U_{j,\ell}$)
b) Append("challenge_prefix", "c_")
c) Append("challenge_index", $\mathsf{b}_8(j)$)
d) $c_j \leftarrow$ ChallengeScalar("challenge_separator").

(4.3) Using the round challenges $c_0, \ldots, c_{d-1}$ and the final witnesses $\boldsymbol{a}_f, \boldsymbol{b}_f$ from the proof, expand back to n_{circuit}-dimensional scalar vectors (Algorithm 4, Phase B, Section 9): $(\boldsymbol{s}_g,\ \boldsymbol{s}_h,\ s_Q,\ s_P,\ \boldsymbol{s}_U)$, where $s_Q = \langle \boldsymbol{a}_f, \boldsymbol{b}_f \rangle$, $s_P = \prod_j c_j^{k-1}$ (the commitment scalar), $\boldsymbol{s}_g, \boldsymbol{s}_h$ are the expanded generator scalars, and $\boldsymbol{s}_U$ are the cross-term scalars.

(4.4) Compute Phase-1 generator-scalars: The following computations combine the IPA output with the circuit's weight vectors $(\boldsymbol{w}_L, \boldsymbol{w}_R, \boldsymbol{w}_O, \boldsymbol{w}_V$ from Verification 3) to produce the generator coefficients that enter the Mega-MSM. They correspond to the re-weighted generators $\boldsymbol{h}' = \boldsymbol{h}^{\boldsymbol{y}^{-n}}$ and the constraint multipliers of [1, Eqs. 81–85]:

a) $y^{-1} \leftarrow y.\mathsf{Invert}()$ and y^{-i} for $i = 0, \ldots, n_{\text{circuit}} - 1$ (a value s is invertible if and only if $s \neq 0$, which was verified in check (2.B)).
b) $w'_R[i] \leftarrow w_{R,i} \cdot y^{-i}$ for $i < n_c$, zero-padded to length n_{circuit}.
c) $\delta \leftarrow \langle \boldsymbol{w}'_R[0 \ldots n_c{-}1], \boldsymbol{w}_L \rangle$ (the cross-term correction that appears in the polynomial check, [1, Eq. 86]).
d) For $i = 0, \ldots, n_{\text{circuit}} - 1$:

$$g_{\text{scalar}}[i] \ \leftarrow\ s_{g,i} \ -\ x \cdot w'_{R,i} \cdot s_P, \tag{26}$$

   where $w'_{R,i} = w_{R,i}\, y^{-i}$ for $i < n_c$ and $w'_{R,i} = 0$ for $i \geq n_c$ (step 4.4b).
e) For $i = 0, \ldots, n_{\text{circuit}} - 1$ (with $w_{L,i} = w_{O,i} = 0$ for $i \geq n_c$):

$$h_{\text{scalar}}[i] \ \leftarrow\ y^{-i} s_{h,i} \ +\ \Big(y^{-i}\big(x\, w_{L,i} + w_{O,i}\big) - 1\Big) \cdot (-s_P) \ +\ y^{-i}\, x_{\text{ipp}}\, (-s_P)\, w_{V,i}. \tag{27}$$

   Eqs. (26)–(27) are identical to Eqs. 30–31 of §11.15 and can be validated entry-by-entry against those test vectors.

These g_{scalar} and h_{scalar} vectors are the per-generator scalars contributed by the Phase-1 (IPA) equation E_0 to the Mega-MSM (Section 8.6). They will be combined with the eCP contribution in Verification 5.

Verification 4 (IPA and Phase-1 Scalar Assembly)

Required checks:

(4.A) Every IPA cross-term point decompresses successfully (i.e., $\mathsf{Decompress}(U_{j,\ell})$ does not return $\perp$).

(4.B) Every round challenge satisfies $c_j \neq 0$ (a zero challenge cannot be inverted in the scalar expansion).

(4.C) $|\boldsymbol{a}_f| = |\boldsymbol{b}_f| = \ell_d$ (the base-case size from Algorithm 1, Section 4.6).

Variable	Description	Section
c_j	IPA round challenges	§B.3
$\boldsymbol{s}_g, \boldsymbol{s}_h$	Generator scalars	§9
s_Q, s_P	Value and commitment scalars	§9
$\boldsymbol{s}_U$	Cross-term scalars	§9

8.5 eCP Sub-Protocol

This verification handles Phase 2 of the MAYA protocol: the *extended Chaum–Pedersen* (eCP) argument that the same permutation committed in V was applied to the ciphertexts ([1, Eqs. 93–114]). It also derives the batching scalar r and assembles the combined scalars B_s and B_b that collect per-equation contributions to the Pedersen generators B and B_{blinding}.

Variable	Description	Section
$y, z, x, x_{\text{ipp}}, w_{\text{agg}}$	Main challenges	§6.4
δ	$\langle \boldsymbol{w}'_R, \boldsymbol{w}_L \rangle$	this section
$t_x, \tau_x, \mu, t_{c,x}, \tau_{c,x}, \mu_c, t_{\text{cross}}, \rho_c$	Proof header scalars	§4.3
α_{ecp}	Batches ElGamal coordinates	§6.4
$\boldsymbol{z}_s, s_{P,e}, \boldsymbol{s}_A$	eCP outputs	§10
r	Batching scalar	§6.4
B_s, B_b	Combined for B, B_{blinding}	this section

Verification 5 (eCP and Batching Setup)

Required computations:

(5.1) Compute polynomial evaluation aggregate $\hat{t} \leftarrow t_x + x_{\text{ipp}} \cdot t_{\text{cross}} + x_{\text{ipp}}^2 \cdot t_{c,x}$ (the "expected inner product" that combines the Phase-1 evaluation t_x, the cross-phase term t_{cross}, and the bridging evaluation $t_{c,x}$, weighted by the aggregation challenge x_{ipp}; see [1, Eqs. 86, 112]).

(5.2) Compute eCP batching challenge $\alpha_{\text{ecp}} \leftarrow \mathcal{T}.\text{ChallengeScalar}(\texttt{"chall\_batched\_ecp"})$ (schedule position after the IPA sub-transcript, Section 6.4). This challenge batches the two ElGamal coordinate equations into a single eCP row ([1, Section 2.4])[a].

(5.3) Replay the eCP sub-protocol on the same transcript $\mathcal{T}$, following the same structure as the IPA replay in step 4.1–4.3 but with different labels:

a) $\text{Append}(\texttt{"protocol-name"}, \texttt{"ecp\_fold"})$
b) $\text{Append}(\texttt{"n"}, \mathsf{b}_8(n_{\text{circuit}}))$
c) $\text{Append}(\texttt{"k"}, \mathsf{b}_8(k))$
d) For each round $j = 0, \ldots, d-1$ and each cross-term index $\ell = 0, \ldots, 2k-3$:
   - $\text{Append}(\texttt{"A\_round"}, \mathsf{b}_8(j))$
   - $\text{Append}(\texttt{"A\_index"}, \mathsf{b}_8(\ell))$
   - $\text{CommitPoint}(\texttt{"A\_point\_0"}, A_{j,\ell}^{(0)})$
   - $\text{CommitPoint}(\texttt{"A\_point\_1"}, A_{j,\ell}^{(1)})$
e) $\text{Append}(\texttt{"challenge\_prefix"}, \texttt{"c\_"})$
f) $\text{Append}(\texttt{"challenge\_index"}, \mathsf{b}_8(j))$
g) $c_j \leftarrow \text{ChallengeScalar}(\texttt{"challenge\_separator"})$.
h) Run the eCP scalar expansion (Algorithm 5, Section 10) to obtain $(\boldsymbol{z}_s,\ s_{P,e},\ \boldsymbol{s}_A)$.

(5.4) Compute deterministic batching scalar $r \leftarrow \mathcal{T}.\text{ChallengeScalar}(\texttt{"batching\_r"})$. This is the last transcript operation. The scalar r is derived deterministically from the transcript state, Because r is derived after every proof element has entered the transcript, no adversary can predict its value when constructing the proof.

(5.5) Combine Pedersen-generator scalars: The scalars B_s and B_b collect contributions from the IPA folding equation (denoted E_0), the polynomial check (denoted E_1, [1, Eq. 86]), and the bridging check (denoted E_2, [1, Eq. 112]) into a single coefficient for the Pedersen generators B and B_{blinding} respectively:

$$B_s \leftarrow w_{\text{agg}}\, s_Q - w_{\text{agg}} \cdot \hat{t} \cdot s_P + r\Big(x^2(w_c + \delta) - t_x\Big) - r^2\, t_{c,x} \tag{28}$$

$$B_b \leftarrow x_{\text{ipp}}\, \mu_c\, s_P + \mu\, s_P - r^2\, \tau_{c,x} - r\, \tau_x \tag{29}$$

where μ and μ_c are the blinding scalars read from the proof header (Section 4.3). The first line of each formula is the E_0 contribution (IPA); the terms multiplied by r are from E_1 (polynomial check); the terms multiplied by r^2 are from E_2 (bridging check).

[a] *Ordering:* α_{ecp} is derived from the main transcript *after* the IPA sub-transcript completes but *before* the eCP sub-transcript begins. The batching scalar r is derived *after* the eCP sub-transcript. The timeline is:

1. Main transcript steps #0–#33 (Verification 2)
2. IPA sub-transcript (Verification 4)
3. $\alpha_{\text{ecp}} \leftarrow \text{ChallengeScalar}(\texttt{"chall\_batched\_ecp"})$
4. eCP sub-transcript (this step)
5. $r \leftarrow \text{ChallengeScalar}(\texttt{"batching\_r"})$

Verification 5 (eCP and Batching Setup)

Required checks:
(5.A) Every eCP cross-term point (both points of every pair) decompresses successfully, and every round challenge satisfies $c_j^e \neq 0$.
(5.B) $\alpha_{\text{ecp}} \neq 0$ and $r \neq 0$.

8.6 Batched Verification Equations and Mega-MSM

The verifier does not check each equation independently. Instead, it scales the five verification equations by consecutive powers of the deterministic batching scalar r (step 5.4) and sums them into a single multi-scalar multiplication, called the *Mega-MSM* (following the technique of [5, Section 4.2] and [1, Section 2.5]).

8.6.1 The Five Batched Equations

Each equation has the form $\sum_j s_j P_j = \mathcal{O}$ (the group identity, i.e., the unique Ristretto255 point whose compressed encoding is 32 zero bytes). The table below lists every equation, its batching weight, the MAYA paper reference, and a one-line formula showing the key relation it checks.

Weight	Equation	Paper ref.	What it checks
r^0	E_0: IPA folding	[1, Eq. 85, 87]; [5, Eq. 68]	$P = A_I^x \cdot A_O^{x^2} \cdots S^{x^3}$ with $\hat{t} = \langle \boldsymbol{a}_f, \boldsymbol{b}_f \rangle$. The folded witness is consistent with the wire commitments and the inner product equals the claimed value.
r^1	E_1: polynomial	[1, Eq. 86]	$g^{t_x} h^{\tau_x} \stackrel{?}{=} g^{x^2(\delta + w_c)} \cdot \prod_{i=1}^{6} T_i^{x^i}$. The evaluation t_x matches the committed polynomial $t(X)$.
r^2	E_2: bridging	[1, Eq. 112]	$g^{t_{c,x}} h^{\tau_{c,x}} \stackrel{?}{=} T_2 \cdot T_1'^{x'}$. The cross-phase evaluation $t_{c,x}$ is consistent with the bridging commitment.
r^3	E_3: eCP ciphertext	[1, Eqs. 3, 114]	The folded eCP witness $\boldsymbol{z}$ satisfies the aggregated ciphertext relation: the output ciphertexts $\boldsymbol{C'}$ weighted by $\boldsymbol{z}_s$ match the inputs $\boldsymbol{C}$ weighted by e_i, batched via α_{ecp}.
r^4	E_4: eCP commitment	[1, Eq. 113]	The same folded witness opens V: $V \cdot S'^{x'} = \boldsymbol{g}^{\boldsymbol{z}_s} \cdot B_{\text{blinding}}^{\mu_c}$, ensuring Phase 2 uses the same permutation as Phase 1.

Write the five equations as $E_0 = \mathcal{O}, \ldots, E_4 = \mathcal{O}$. The Mega-MSM computes $R = E_0 + rE_1 + r^2E_2 + r^3E_3 + r^4E_4$ [5]. If every $E_j = \mathcal{O}$, then $R = \mathcal{O}$ for any r. Conversely, if some $E_j \neq \mathcal{O}$, then R is a non-zero polynomial of degree ≤ 4 in r with group-element coefficients. Because r is a deterministic hash of the entire transcript (and thus fixed only *after* all proof data has been committed), it is heuristically uniform in $\mathbb{Z}_p$ (in the random-oracle model). By the Schwartz–Zippel lemma [9], the probability that $R = \mathcal{O}$ despite some $E_j \neq \mathcal{O}$ is at most $4/p < 2^{-250}$. This batching technique originates in [5, Section 4.2] and is analyzed for MAYA in [1, Section 2.5].

A successful identity check therefore confirms: (1) the committed vector $\boldsymbol{e'}$ is a permutation of $\boldsymbol{e}$, (2) the ciphertext transformation uses that same permutation, (3) all polynomial and folding equations are satisfied, and hence (4) $\boldsymbol{C'}$ is a valid shuffle of $\boldsymbol{C}$.

[5] The aggregation challenge w_{agg} is *not* a batching weight: it is fixed *inside* E_0 as the coefficient of the value base, $Q = B^{w_{\text{agg}}}$, and therefore appears only in the B-point scalar B_s of Eq. (28); see also §11.16.

8.6.2 Mega-MSM: Point and Scalar Table

The table below gives every (scalar, point) pair that enters the Mega-MSM, in the order they are assembled. Throughout: $s_P = \prod_j c_j^{k-1}$ (IPA product scalar), $s_{P,e}$ (eCP product scalar, Section 10), $r_j = r^j$, $\alpha = \alpha_{\text{ecp}}$, and $i = 1, \ldots, n$.

#	**Point**	**Scalar**
1	A_I	$-x \cdot s_P$
2	A_O	$-x^2 \cdot s_P$
3	S	$-x^3 \cdot s_P$
4	V	$-x_{\text{ipp}}\, s_P + r_4\,(-s_{P,e})$
5	S'	$-x_{\text{ipp}}\, x'\, s_P + r_4\, x'\,(-s_{P,e})$
6	B	$B_s + s_{P,e}\, r_3\, \rho_c$ (Eq. 28)
7	B_{blinding}	$B_b + s_{P,e}\, r_4\, \mu_c$ (Eq. 29)
8	F	$s_{P,e}\, r_3\, \alpha\, \rho_c$
9–8+n_{circuit}	$g_1, \ldots, g_{n_{\text{circuit}}}$	$g_{\text{scalar}}[i] + z_{s,i} \cdot r_4$ (step 4.4d + eCP)
next n_{circuit}	$h_1, \ldots, h_{n_{\text{circuit}}}$	$h_{\text{scalar}}[i]$ (step 4.4e)
next $d(2k-2)$	$U_{j,\ell}$ (IPA cross-terms)	$-s_{U,j,\ell}$
next	T'_1	$r_2\, x'$
next	T_2	r_2
next 6	$T_1, T_2, T_3, T_4, T_5, T_6$	$rx,\ rx^2,\ rx^3,\ rx^4,\ rx^5,\ rx^6$
next	S'_1	$r_3\, x'\,(-s_{P,e})$
next	S'_2	$r_3\, \alpha\, x'\,(-s_{P,e})$
next n	$C_{1,1}, \ldots, C_{n,1}$	$e_i \cdot r_3\,(-s_{P,e})$[6]
next n	$C_{1,2}, \ldots, C_{n,2}$	$e_i \cdot r_3\, \alpha\,(-s_{P,e})$
next n	$C'_{1,1}, \ldots, C'_{n,1}$	$z_{s,i} \cdot r_3$
next n	$C'_{1,2}, \ldots, C'_{n,2}$	$z_{s,i} \cdot r_3 \cdot \alpha$
next $d(2k-2)$	$A^{(0)}_{j,\ell}$ (eCP row-0)	$-s_{A,j,\ell} \cdot r_4$
next $d(2k-2)$	$A^{(1)}_{j,\ell}$ (eCP row-1)	$-s_{A,j,\ell} \cdot r_3$

Total rows: $18 + 2\, n_{\text{circuit}} + 4n + 3\, d(2k-2)$. The point T_2 occupies two rows (scalars r^2 and $r\, x^2$); implementations that pre-accumulate per distinct point therefore pass $17 + 2\, n_{\text{circuit}} + 4n + 3\, d(2k-2)$ points to the MSM, with the T_2 scalar equal to $r^2 + r\, x^2$. For the test instance ($n = n_{\text{circuit}} = 4$, $k = 2$, $d = 2$): 54 rows, 53 distinct points.

[6] Only the first n entries of $\boldsymbol{z}_s$ multiply the ciphertext components; the full n_{circuit}-length vector multiplies the $\boldsymbol{g}$ generators (row block 9).

8.7 Verification 6: Final Identity Check

Verification 6 (Mega-MSM Identity Check)

Required computations:

(6.1) Assemble $\boldsymbol{s}_{\text{all}}$ and $\boldsymbol{P}_{\text{all}}$ from the table in Section 8.6.2.
(6.2) Compute $R \leftarrow \text{MSM}(\boldsymbol{s}_{\text{all}}, \boldsymbol{P}_{\text{all}}) = \sum_j s_j \cdot P_j$.

Implementation note. Some MSM terms may have scalar value 0 (e.g., when a weight vector entry is zero or a cross-term scalar reduces to zero for a particular challenge). Mathematically, $0 \cdot P = \mathcal{O}$ and contributes nothing to the sum. Implementations may either: (a) skip zero-scalar entries before calling the MSM library, or (b) use an MSM implementation that handles zero scalars internally. Note that libsodium's `crypto_scalarmult_ristretto255` rejects scalar 0 with an error code; implementations using libsodium must choose option (a).

Required checks:

(6.A) $|\boldsymbol{s}_{\text{all}}| = |\boldsymbol{P}_{\text{all}}|$.
(6.B) $R = \mathcal{O}$ (the group identity; for Ristretto255, Compress(R) is 32 zero bytes).

If $R = \mathcal{O}$, the proof is Accepted. Otherwise, the proof is Rejected.

9 IPA Verification Sub-Protocol

This section specifies the inner-product-argument (IPA) verification-scalars computation (Algorithm 4) called by Verification 4 (Section 8.4). Given the IPA sub-proof $(k, d, m, \{U_{j,\ell}\}, \boldsymbol{a}_f, \boldsymbol{b}_f)$ (parsed in Section 4.4) and the shared main transcript $\mathcal{T}$, the algorithm produces five outputs that enter the Mega-MSM (Section 8.6.2):

Output	Meaning
$\boldsymbol{s}_g \in \mathbb{Z}_p^{n_{\text{circuit}}}$	Expanded scalars for the $\boldsymbol{g}$-generator vector. Combined with the circuit weights in step (4.4d).
$\boldsymbol{s}_h \in \mathbb{Z}_p^{n_{\text{circuit}}}$	Expanded scalars for the $\boldsymbol{h}$-generator vector. Combined with the circuit weights in step (4.4e).
$s_Q \in \mathbb{Z}_p$	Inner product $\langle \boldsymbol{a}_f, \boldsymbol{b}_f \rangle$ (the claimed evaluation $\hat{t}$; enters the B-scalar in step (5.5)).
$s_P \in \mathbb{Z}_p$	Product $\prod_j c_j^{k-1}$ (multiplies the commitment scalars in steps (4.4d), (4.4e), (5.5)).
$\boldsymbol{s}_U \in \mathbb{Z}_p^{d(2k-2)}$	Cross-term scalars for the U-points (negated in the Mega-MSM, Section 8.6.2).

The sub-protocol appends to the *same* main transcript $\mathcal{T}$ that Verification 2 initialized (§8.2). The transcript labels are listed in Appendix B.3. The iterative padding that determines the round lengths $(\ell_0, \ell_1, \ldots, \ell_d)$ is specified by Algorithm 1 (Section 4.6), with input $(n_{\text{circuit}}, k, d)$.

The algorithm has two phases. Phase A replays the prover's transcript operations (committing cross-term points and deriving round challenges); this is the same sequence that Verification 4 (§8.4), steps (4.1)–(4.2) describe at a high level. Phase B expands the final witness back to n_{circuit} dimensions.

Algorithm 4 IPA verification scalars (Part I)

Input: n_{circuit} (§5), sub-proof $(k, d, m, \{U_{j,\ell}\}, \boldsymbol{a}_f, \boldsymbol{b}_f)$ (§4.4), transcript $\mathcal{T}$.
Output: $(\boldsymbol{s}_g,\ \boldsymbol{s}_h,\ s_Q,\ s_P,\ \boldsymbol{s}_U)$.

Phase A: transcript replay and challenge derivation

$\mathcal{T}$.Append("protocol-name", "ipa_fold")
$\mathcal{T}$.Append("n", $\mathsf{b}_8(n_{\text{circuit}})$)
$\mathcal{T}$.Append("k", $\mathsf{b}_8(k)$)
$(\ell_0, \ldots, \ell_d) \leftarrow$ ReconstructRoundLengths$(n_{\text{circuit}}, k, d)$ ▷ Algorithm 1, §4.6
Verify $m = \ell_d$ and $|\boldsymbol{a}_f| = |\boldsymbol{b}_f| = m$ ▷ reject on mismatch
for $j = 0, 1, \ldots, d-1$ **do**
    for $\ell = 0, 1, \ldots, 2k-3$ **do**
        $\mathcal{T}$.Append("U_round", $\mathsf{b}_8(j)$)
        $\mathcal{T}$.Append("U_index", $\mathsf{b}_8(\ell)$)
        $\mathcal{T}$.CommitPoint("U_point", $U_{j,\ell}$)
    end for
    $\mathcal{T}$.Append("challenge_prefix", "c_")
    $\mathcal{T}$.Append("challenge_index", $\mathsf{b}_8(j)$)
    $c_j \leftarrow \mathcal{T}$.ChallengeScalar("challenge_separator") ▷ reject if $c_j = 0$
end for

Phase B: scalar computation

Compute the inverses $c_0^{-1}, \ldots, c_{d-1}^{-1}$ ▷ batch inversion
$s_P \leftarrow \prod_{j=0}^{d-1} c_j^{k-1}$

— **Expand $\boldsymbol{s}_g$ from $\boldsymbol{a}_f$** —

$\boldsymbol{s}_g \leftarrow \boldsymbol{a}_f$ ▷ start with the m-dimensional final witness
for $j = d-1$ downto 0 **do**
    Compute the k-block $\beta = (1,\ c_j^{-1},\ c_j^{-2},\ \ldots,\ c_j^{-(k-1)})$
    $\boldsymbol{v} \leftarrow$ empty vector
    for $b = 0, 1, \ldots, k-1$ **do**
        for $i = 0, 1, \ldots, |\boldsymbol{s}_g|-1$ **do**
            append $\beta[b] \cdot \boldsymbol{s}_g[i]$ to $\boldsymbol{v}$
        end for
    end for
    $\boldsymbol{s}_g \leftarrow \boldsymbol{v}$
    Truncate $\boldsymbol{s}_g$ to length ℓ_j ▷ discard padding positions
end for
$\boldsymbol{s}_g[i] \leftarrow \boldsymbol{s}_g[i] \cdot s_P$ for all i

— **Expand $\boldsymbol{s}_h$ from $\boldsymbol{b}_f$** —

Same as $\boldsymbol{s}_g$ expansion, but use $\boldsymbol{b}_f$ as the starting vector, the k-block uses *positive* powers $\beta = (1,\ c_j,\ c_j^2,\ \ldots,\ c_j^{k-1})$, and do **not** multiply by s_P at the end.

— **Value-base scalar** —

$s_Q \leftarrow \langle \boldsymbol{a}_f, \boldsymbol{b}_f \rangle$

Algorithm 4 IPA verification scalars (continued)

— **Cross-term scalars** —

33: Define $\text{suffix}_j \leftarrow \prod_{j'>j} c_{j'}^{k-1}$. For each round $j = 0, \ldots, d-1$, produce $2(k-1)$ cross-term scalars in two groups:

- For $\ell = 1, \ldots, k-1$ ("left" cross-terms): $s_U = c_j^{k-1-\ell} \cdot \text{suffix}_j$.
- For $\ell = 1, \ldots, k-1$ ("right" cross-terms): $s_U = c_j^{k-1+\ell} \cdot \text{suffix}_j$.

The scalars are stored in round-major order: all $2(k-1)$ entries for round 0 first (left group then right group), then round 1, etc. The cross-term scalars enter the Mega-MSM *negated*: $-s_{U,j,\ell}$ (Section 8.6.2).

Implementation note. If the expanded scalars do not match the test vectors in §11, check the following common pitfalls:

1. In Algorithm 4 (lines 22–23), the block multiplier index b *must* be the outer loop, and the element index i must be the inner loop. For each multiplier $\beta[b]$, you iterate over all current entries. This produces the ordering $(\beta[0] \cdot v_0,\ \beta[0] \cdot v_1,\ \beta[1] \cdot v_0,\ \beta[1] \cdot v_1)$.

   For example, with $k = 2$, a starting vector (a, b), and $\beta = (1, c^{-1})$:

   - **Correct** (*b-outer*): $(a,\ b,\ ac^{-1},\ bc^{-1})$
   - **Wrong** (*i-outer*): $(a,\ ac^{-1},\ b,\ bc^{-1})$

   The single-element worked example in §9 does not distinguish these orderings because both produce the same result when $|\boldsymbol{s}_g| = 1$. If $\mathtt{sg}[1]$ and $\mathtt{sg}[2]$ test vectors are swapped, the loop order is reversed.

2. Ensure that $\boldsymbol{s}_g$ is multiplied by s_P at the end of the expansion, but $\boldsymbol{s}_h$ is *not*.

3. Check that the $\boldsymbol{s}_g$ expansion uses *inverse* challenges (c_j^{-1}), while the $\boldsymbol{s}_h$ expansion uses *positive* challenges (c_j).

4. Verify $s_P = \prod_j c_j^{k-1}$ independently by computing it directly from the test-vector challenges and comparing.

Example ($k = 2$, $d = 2$, $m = 1$)

Let $k = 2$, $d = 2$, $m = 1$, $n_{\text{circuit}} = 4$. The round lengths from Algorithm 1 (Section 4.6) are $(\ell_0, \ell_1, \ell_2) = (4, 2, 1)$. Let the round challenges be c_0, c_1.

Expanding $\boldsymbol{s}_g$ from $\boldsymbol{a}_f = (a)$.

1. Round $j = 1$: block $\beta = (1, c_1^{-1})$. $\boldsymbol{s}_g = (a,\ a \cdot c_1^{-1})$. Truncate to $\ell_1 = 2$: no change.
2. Round $j = 0$: block $\beta = (1, c_0^{-1})$. $\boldsymbol{s}_g = (a,\ ac_0^{-1},\ ac_1^{-1},\ ac_0^{-1}c_1^{-1})$. Truncate to $\ell_0 = 4$: no change.
3. Multiply by $s_P = c_0 \cdot c_1$: $\boldsymbol{s}_g = (ac_0c_1,\ ac_1,\ ac_0,\ a)$.

Cross-term scalars. With $k = 2$ there are $2(k-1) = 2$ cross-terms per round (one left, one right):

- Round $j = 0$: $\text{suffix}_0 = c_1^{k-1} = c_1$. Left ($\ell = 1$): $c_0^{k-1-1} \cdot c_1 = c_0^0 \cdot c_1 = c_1$. Right ($\ell = 1$): $c_0^{k-1+1} \cdot c_1 = c_0^2 \cdot c_1$.
- Round $j = 1$: $\text{suffix}_1 = 1$. Left ($\ell = 1$): $c_1^0 = 1$. Right ($\ell = 1$): c_1^2.

These can also be compared against the test vectors for $n = 4$ (Section 11).

10 eCP Verification Sub-Protocol

This section specifies the extended-Chaum–Pedersen (eCP) verification-scalars computation (see Algorithm 5) called by Verification 5, step (5.3) in Section 8.5. Given the eCP sub-proof

$(k, d, m, \{A_{j,\ell}\}, \boldsymbol{z})$ (parsed in Section 4.5), the algorithm produces three outputs:

Output	Meaning
$\boldsymbol{z}_s \in \mathbb{Z}_p^{n_{\text{circuit}}}$	Expanded scalars for the $\boldsymbol{g}$ generators and ciphertext components in the Mega-MSM.
$s_{P,e} \in \mathbb{Z}_p$	Product $\prod_j (c_j^e)^k$; multiplies eCP commitment and ciphertext scalars.
$\boldsymbol{s}_A \in \mathbb{Z}_p^{d(2k-2)}$	Cross-term scalars for the A-point pairs (negated in the Mega-MSM, split by row).

Algorithm 5 operates on the *shared* main transcript $\mathcal{T}$, continuing from the state left by Algorithm 4 and by the derivation of α_{ecp} at schedule step #34 (§6.4. Note that α_{ecp} is derived *before* the `Append("protocol-name", "ecp_fold")` of this algorithm, even though it is only *used* in the eCP batching equations. Its labels are in Appendix B.4.

The algorithm has the same two-phase structure as the IPA (Algorithm 4): transcript replay followed by scalar expansion. The following table summarizes the differences; everything not listed is identical to the IPA algorithm. Differences marked ($\star$) cause silent errors if missed (e.g., the final MSM check will simply fail to produce the identity element with no indication of which difference was overlooked).

Aspect	IPA	eCP	
Protocol name	`"ipa_fold"`	`"ecp_fold"`	
Cross-term structure	Single point $U_{j,\ell}$; label `"U_point"`	Point *pair* $[A_{j,\ell}^{(0)}, A_{j,\ell}^{(1)}]$; labels `"A_point_0"`, `"A_point_1"`	
Product scalar	$\prod_j c_j^{k-1}$	$\prod_j (c_j^e)^k$	($\star$)
Expanded vectors	Two: $\boldsymbol{s}_g$ (from $\boldsymbol{a}_f$) and $\boldsymbol{s}_h$ (from $\boldsymbol{b}_f$)	One: $\boldsymbol{z}_s$ (from $\boldsymbol{z}$)	
Final s_P multiplication	$\boldsymbol{s}_g$ multiplied by s_P; $\boldsymbol{s}_h$ is **not**	$\boldsymbol{z}_s$ **is** multiplied by $s_{P,e}$	($\star$)
Cross-term exponents	left: $c_j^{k-1-\ell}$; right: $c_j^{k-1+\ell}$	left: $(c_j^e)^{\ell}$; right: $(c_j^e)^{k+\ell}$	($\star$)
Suffix product	$\prod_{j'>j} c_{j'}^{k-1}$	$\prod_{j'>j} (c_{j'}^e)^k$	($\star$)
MSM row split	$-s_{U,j,\ell}$ (single row)	row-0: $-s_A \cdot r^4$; row-1: $-s_A \cdot r^3$	

11 Test Vectors

This section provides reference values for every intermediate computation that a MAYA verifier performs. An implementer can validate each stage of the verifier independently by comparing its output against the values below before moving to the next stage, following the incremental build plan in Section 1.3.

11.1 Purpose and Scope

The test vectors serve three main roles:

Algorithm 5 eCP verification scalars

Input: n_{circuit}, eCP sub-proof $(k_e, d_e, m_e, \{A_{j,\ell}\}, \boldsymbol{z})$ (§4.5), transcript $\mathcal{T}$
Output: $(\boldsymbol{z}_s,\ s_{P,e},\ \boldsymbol{s}_A)$.

Phase A: transcript replay

$\mathcal{T}$.Append("protocol-name", "ecp_fold")
$\mathcal{T}$.Append("n", $\mathsf{b}_8(n_{\text{circuit}})$)
$\mathcal{T}$.Append("k", $\mathsf{b}_8(k_e)$)
$(\ell_0, \ldots, \ell_{d_e}) \leftarrow$ ReconstructRoundLengths$(n_{\text{circuit}}, k_e, d_e)$
Verify $|\boldsymbol{z}| = \ell_{d_e}$
for $j = 0, 1, \ldots, d_e - 1$ **do**
    for $\ell = 0, 1, \ldots, 2k_e - 3$ **do**
        $\mathcal{T}$.Append("A_round", $\mathsf{b}_8(j)$)
        $\mathcal{T}$.Append("A_index", $\mathsf{b}_8(\ell)$)
        $\mathcal{T}$.CommitPoint("A_point_0", $A^{(0)}_{j,\ell}$)
        $\mathcal{T}$.CommitPoint("A_point_1", $A^{(1)}_{j,\ell}$)
    end for
    $\mathcal{T}$.Append("challenge_prefix", "c_")
    $\mathcal{T}$.Append("challenge_index", $\mathsf{b}_8(j)$)
    $c^e_j \leftarrow \mathcal{T}$.ChallengeScalar("challenge_separator") ▷ reject if $c^e_j = 0$
end for

Phase B: scalar computation

$s_{P,e} \leftarrow \prod_{j=0}^{d_e-1} (c^e_j)^{k_e}$ ▷ exponent is k_e (IPA uses $k-1$)

— **Expand $\boldsymbol{z}_s$ from $\boldsymbol{z}$** —

Same interleave-and-truncate expansion as $\boldsymbol{s}_g$ in Algorithm 4, using $\boldsymbol{z}$ as the starting vector, inverse challenges $(c^e_j)^{-1}$, and round lengths $(\ell_0, \ldots, \ell_{d_e})$.
$\boldsymbol{z}_s[i] \leftarrow \boldsymbol{z}_s[i] \cdot s_{P,e}$ for all i ▷ unlike $\boldsymbol{s}_h$ in IPA, $\boldsymbol{z}_s$ is multiplied by $s_{P,e}$

— **Cross-term scalars** —

Define $\text{suffix}_j = \prod_{j'>j} (c^e_{j'})^{k_e}$. For each round j, produce $2(k_e-1)$ cross-term scalars in two groups:

- For $\ell = 1, \ldots, k_e-1$: $s_A = (c^e_j)^{\ell} \cdot \text{suffix}_j$.
- For $\ell = 1, \ldots, k_e-1$: $s_A = (c^e_j)^{k_e+\ell} \cdot \text{suffix}_j$.

The cross-term scalars enter the Mega-MSM *negated* and split by row: row-0 points $A^{(0)}_{j,\ell}$ carry scalar $-s_{A,j,\ell} \cdot r^4$, and row-1 points $A^{(1)}_{j,\ell}$ carry $-s_{A,j,\ell} \cdot r^3$ (Section 8.6.2).

1. Stage-by-stage debugging. Each subsection below corresponds to one step in the implementation plan (Section 1.3). A developer implementing the verifier from scratch can compare their output against the reference after each step: CRS generators (§11.5–11.6), challenge vector (§11.7), transcript challenges (§11.8), constraint weights (§11.9), proof parsing (§11.10–11.12), and the final identity check (§11.17). A mismatch at any stage pinpoints the module that needs fixing.

2. Formula disambiguation. Sections 11.16–11.8 provide explicit intermediate scalars in operator grouping in the formulas of Sections 7–9.

3. Conformance testing. Two conforming implementations that process the same input bytes must produce identical outputs at every stage. The values below constitute a conformance suite: if an implementation reproduces them exactly, it will agree with the reference verifier on all inputs (assuming no further bugs in post-test-vector logic).

4. Positive verification. The test proof was generated from a known-correct shuffle (the fixed permutation $\pi = (1, 3, 0, 2)$ with deterministic re-randomization). The expected final result is Accept ($R = \mathcal{O}$, Section 11.17).

11.2 How These Values Were Generated

Determinism and Reproducibility. The verifier is deterministic, since it only evaluates public computations together with the hash-based transcript. To ensure that the prover outputs are byte-identical and fully reproducible on every run, no operating-system entropy was used. Both the transcript (HMAC-SHA-256) and the prover's internal random-number generator were initialized with fixed deterministic seeds. Consequently, re-running the verifier on the exact same inputs must always produce an Accept decision.

Test Case Parameters. The test vectors represent a shuffle instance with the following dimensions: $n = 4$, $k = 2$, $d = 2$, and $m = 1$. The Bulletproof generator set has a capacity of $N = n + k - 1 = 5$ (Section 5.2). The applied permutation is $\pi = (1, 3, 0, 2)$.

Reference Implementation and Seeds. All values were generated by the HMAC-SHA-256 reference implementation of MAYA [2], using the `generate_test_vectors` function located in `tests/test_vectors.rs`. The inputs and blinding factors were generated using the `det_scalar` function with the following hardcoded seeds:

- Input Ciphertexts: $\mathbf{c}_1$ uses scalar indices 100–103 applied to B; $\mathbf{c}_2$ uses scalar indices 200–203 applied to F.
- Re-randomization: Scalars use indices 300–303.
- Commitment Blinding: Uses seed 999.

11.3 Encoding Conventions

All values are printed as 64 hexadecimal characters encoding 32 bytes, *in byte order*: the first two hex characters are byte 0, the next two are byte 1, and so on. For scalars, byte 0 is the *least* significant byte (canonical little-endian, §4.1). Group elements are canonical Ristretto255 compressed encodings (§4.1).

11.4 Input Data

Input ciphertexts, shuffled ciphertexts, and output commitment. Compare after implementation steps 1–2. $C_1[i] = B \cdot \mathsf{det\_scalar}(i + 100)$, $C_2[i] = F \cdot \mathsf{det\_scalar}(i + 200)$, $C'_1[j] = C_1[\pi(j)] + B \cdot \mathsf{det\_scalar}(j + 300)$, $C'_2[j] = C_2[\pi(j)] + F \cdot \mathsf{det\_scalar}(j + 300)$ with $\pi = (1, 3, 0, 2)$.

```
C1[0]  = d8da78b072acc4701859b2eac090f15da39dae7c3972828163d0d077cea2c93f
C2[0]  = 4c9477a21cddce00c4d3354ceac31d6814a76b804bfd30ea4526aa52c122f776
C1[1]  = c04556183d635aefb4d99cabadd1a33768dd4f47acfec578f20793ce6d847864
C2[1]  = 32aa97edfb23507d5b2c622af8ec600e83c4c3f94eb4d6c6b89b128483d40f5c
C1[2]  = 7a667ef103c4ebe23a1e578b41e21b153e2862b86dec042fef231d99e306f13b
C2[2]  = 0af754ed6a41a8776b4337c0d0e0cbe416c07303e1a90b868b7024dddeb7ab70
C1[3]  = b8cfe8048d4f95aa00612425121dd1680f4285b4f1e760c7ea10b0ed04e8522c
C2[3]  = 1ae4184efe15aa60ddcdb956b1a257c230c992275997b9d32e795c121ac9353e
C1'[0] = 7a053a393e2598bddfa6e9fc6a8f27a72a658fc849bb7d205ae5e75ab36b6a1b
C2'[0] = 0c5a9d0c621f40d009e8d76a63df6ac6dc505108de3569c6b6c76cba93fbfa6b
C1'[1] = 704885d6c74a7f3823a0afc50a56194ab69987025bf57f8c34234db415dd642c
C2'[1] = 441120737e4b1632c2718ed4a4479dd40271bb927b943ff9d0ff17a41ff2a65f
C1'[2] = 2ea769936a86ca2af2d0ab7374079b8f5342be7647b5663c93adaf18b0f57255
C2'[2] = 2030041c421ff85b8f5a0b0727676c827d8089e4f24c024a2cce5eb9d0c2bf1e
C1'[3] = 06077dee6d85c25ecfef48e38aeab362fc5e46f4697b9129ae7031c491f06874
C2'[3] = 3cfb81b913718cedae99e85142421cef84d56d33c1224542296787 8f9637f1c
V      = ca33cdd54b2ecc2de905f3148989a01714f47bc6192d671037d5107b05579b60
```

V is the Pedersen vector commitment to the permuted evaluation vector $(\mathbf{e}[\pi^{-1}(0)], \ldots, \mathbf{e}[\pi^{-1}(n-1)])$ with deterministic blinding scalar $\mathsf{det\_scalar}(999)$.

Test Vector Integrity. To guard against transcription errors when extracting hex strings from this document, the SHA-256 digest of the concatenated ciphertext bytes (all 16 values $C_1[0] \| C_2[0] \| C_1[1] \| \cdots \| C'_2[3]$, each decoded from hex to 32 raw bytes, 512 bytes total) is:

```
SHA-256 = 6a89a35f1031463abc80466cac101b1b93a5e3350e7464825d6d4db0d0aca6e4
```

A machine-readable JSON file containing all test vectors is available alongside this specification. If your $\boldsymbol{e}$ values (Section 11.7) do not match, first verify this checksum matches; if not, one or more ciphertext hex strings were corrupted during extraction.

11.5 Pedersen Generators

Derived per Algorithm 1 (Section 5.1). Compare after implementation step 2 of Section 1.3.

```
B          = e2f2ae0a6abc4e71a884a961c500515f58e30b6aa582dd8db6a65945e08d2d76
B_blinding = 94acdcd0ab862f4963d4f2de13996e8038e70728ac49f4b2966c6fea979bcb4a
F          = 884a5327dff12da339bdd577e8eff7a0b0c4ab0ce360884a9136cc91d3ca035a
```

11.6 Bulletproof Generators (first 4)

Derived per Algorithm 2 (Section 5.2) using HMAC-SHA-256 counter mode. Compare after implementation step 2.

```
seed_G = be34c813e72b7774dc77ac06001c2bd3beebb80730225c8df1ae10a003a836dd
seed_H = 0da356bdc9effe908d8f9a072d3acb2ee497358830666cf023cab759eff4e946
g[0]   = e679e7b95edf56c8fd788645d5eaa51030eb7a75af459b6aa0e1743c9dacad14
g[1]   = 9e651dfbe14663535b8d2809b9c10cd575ec5b3c9e049b19119f191a9cda2425
g[2]   = 903040d2100a2a08f554b5da62ced39b505d9cda3f2299d9871018ed7f040b5a
g[3]   = 78d70b0c5637258cf0e3cefb5f8f1cf42023fdfb7cc3663256db1f63dd0f447b
h[0]   = 42ebc43d43de71b8f3cc48532e720ddba14a7bb0961bea66372c66322a05a43e
h[1]   = 628455b1cbd24f72a9845238548b72d455801ee15ab5111dcc320c3f6925f84f
h[2]   = 64f7568cb7413d1c342a0be1a9886c39f6dcfbf3e518c4d8a8dd82ddd2295572
h[3]   = 1aee527988b35d1de08239ada57e93cc8fcd86d7cb213be54d3352a5a0c6201b
```

11.7 Challenge Vector

Derived per Algorithm 3 (Section 7). Compare after implementation step 4. The transcript commits $N = 5$ via `CommitU64("ck_n", 5)` and the CRS seeds via `Append("ck_g_seed", seed_G)` and `Append("ck_h_seed", seed_H)`.

```
e[0] = 0750965ac632de2c6841db6940b583f9e8d9329366803011425503b738f86c0b
e[1] = 206935f9d67646679f04ef9589f63f1334afa8a1defd94345154aeb147b30508
e[2] = 887ddc39c9889848c00df34531f57d0abd3a0b7789ca0ef67f21a6b7cdd34d0f
e[3] = 627cc7f94e7dfc4aa109f2be80e5cfc571057a5926f1953bd5bd61ff1f3ded0c

```

11.8 Transcript Challenges

Derived by replaying the Complete Transcript Schedule (Section 6.4) via the transcript state machine (Section 6.3). Compare after implementation step 3.

```
u                 = fa4931eb90a6f6fa109067f80b7dcdbe0a89dd03c3dbf6757f1486af4e7e520d
y                 = 3039e3cf78c87886b9ca8fa37fb26a50b8d721c115a625dddd2c28c97ce7a707
z                 = fa32422fe3bbb1fa9735ac71906f2c1ecad505012d3ebf04d113ef4432f35609
x                 = dcc4ba0e42027fb0fd07b32d58f02a56536546451c54090bbf63b28686d9ea03
x_prime           = 80dba0268097a855acd2e1e57205a52cadbdedeaf56ca5c27250b3c18330aa06
x_ipp             = 3faebf45711564106fe0b9e81b3f4605c5b8267005e520d68240ac8b49841202
w_agg             = 3f13292b2f9eae5286e43a28f813ee15a9f0990e1b9a0a7f32de9741e043de08
chall_batched_ecp = 2341850eb48c3ebde90070c6d063cff67f09d2d02dec65ad711f4ac26bfaf70b
batching_r        = af7c14a778ec76d7ecbb32544afff0fade208fc0323519b4708044d7cb08b107
```

Note. The shuffle challenge u (schedule step #6, Section 6.4) is derived inside Verification 3 (Section 8.3) before the permutation circuit is built. This occurs after V is committed to the transcript but before the proof points are committed).

11.9 Flattened Weights ($n = 4$)

Computed per the permutation circuit of Section 8.3.1, Eqs. (18)–(22). Compare after implementation step 6.

```
wL[0] = f3a0b32d37a7605d3e674b314e8ab2f6352afafed2c140fb2eec10bbcd0ca906
wL[1] = 740e56355c63367a4034af6bc8d126d9d6ba89c0e07583af42cc046e37950106
wL[2] = 8d5dd7247ef23b71dc22bb431fe03e7fc66a54f268bc3be3f454fdb374e36d03
wR[0] = 858fe047cb94b57f44d4b1994ac450191218240e00232916e9856597ca1da90d
wR[1] = cd1386f1061ba1c4d87c7382654a0e7382e2cad18ea6e605acc200f3ccc97502
wR[2] = df62fdf5748b1d84825062c1d8b545b67583e8e5d22329a9e052410a1bf50c0f
wO[0] = 79c59f27beffdbdd956848371628b83b2945763f1f8a7c50bd33fb91c86afe09
wO[1] = 60761e389c70d6e6f9793c5fbf19a0953995ab0d9743c41c0bab024c8b1c920c
wO[2] = f67dc986bf9e071819085025 0dcd82dd01c97488e2c3af3b2cb90088fb55be0e
wV[0] = f3a0b32d37a7605d3e674b314e8ab2f6352afafed2c140fb2eec10bbcd0ca906
wV[1] = 858fe047cb94b57f44d4b1994ac450191218240e00232916e9856597ca1da90d
wV[2] = cd1386f1061ba1c4d87c7382654a0e7382e2cad18ea6e605acc200f3ccc97502
wV[3] = df62fdf5748b1d84825062c1d8b545b67583e8e5d22329a9e052410a1bf50c0f
wc    = 53cd5ae5e63eccc3f1c1fb85404e3d96596ce7ec81788426fe4fb35090545b03
```

11.10 Proof Header (13 points, 8 scalars)

Parsed per Section 4.3. Compare after implementation step 5.

```
A_I     = 32021443a69eb4562b2b12974737e841a3bd0db31a0d94cafe355bb2d3cb5761
A_O     = 22c5c4cf7559ca790fa3a03d67e1777a8f6e517f0a4c3be3e076ecc59fcce25d
S       = 445e25664116e8400616a64a7bc3487cf2d70e00b29b488dd79ce706f57ae24c
T_1     = 14284ad69a1f7520de7e26a00e8b00c1b62f2bda01a4afc1ed726d4e533e8a53
T_2     = 2ccf55c7c4369fd316c7687a61fa3d2b29dfd8aae379b4c44ad84cfb7c120d29
T_3     = 6a04b23442cbbeb4941724b6facc18efa67ea9568015157e93628e1ec5b2f263
T_4     = f4f36e0a411d5e0e06fadbdba3fdba10b26b79f308ff06a80de11c13b4ad8a2d
T_5     = e2cf1c96eb60765cb661633d8fa9edbc2ef847984490013a16ae5cb49572615b
T_6     = 743b31ae90368be7b6b195803c87268f00db5d4760884759e130525659380b7f
S'      = c8f22511236666913a21f6c1a38aa559f631fa0c3fff44af97d784c43eb10c77
T'_1    = 6091a842c9be7d589e461c1409b0e60c338db786c20decf066ed90371787aa39
```

```
S'_1    = e234f3c78694e19726885a29fc360c05428b7ba2b2d9bab63758b678c2787f3b
S'_2    = d4d86eb31e44511389a639e2f2f8c58dfe659e0ccb8eaae54db23f85c294f52f
t_x     = eaabe383763d6e89d45e759da48f522cc759bd891e82adecb60fdae5e135bf09
tau_x   = 8877fa157e59dac2ac2b25ff911b6db295bab38438bbc67d98d8604662c4af07
mu      = 993940bddecc6cc995ddeca36aedb1d68d592ab2d040165f90187568f35c4508
tc_x    = 46614c90587cd67ecefb0f1ac12d4d9a460d4d05b3b7b0e6a65f1fc65b2e880b
tau_cx  = 302c243ed7c39391c5853b3ed6341faa4fa5009c5613f0be76a2f7f068663f00
mu_c    = 8eb843b23c73424c9b47966e3a0bedbbc189cbe1929b8f3829ef280873334804
rho_c   = 373820c345cecb875dbdda001dced01b063409c2407bbbf2f747e0564fc32908
t_cross = 58aeab99e2dc1de307d7dd445c5c169b09b6f80aca696e3eac9223afd8f5db0b
```

11.11 IPA Sub-Proof ($k = 2$, $d = 2$, $m = 1$)

Parsed per Section 4.4. The cross-term points $U_{j,\ell}$ enter the transcript in Algorithm 4 (Section 9) and the Mega-MSM (Section 8.6.2). With $k = 2$ there are $2(k-1) = 2$ cross-terms per round. Cross-terms enter the Mega-MSM as described in §11.13.

```
U[0][0]  = 82e73124557f6a237d2a0d7f9a6b1612274d92e93fa5c1b32e2f41f86b113915
U[0][1]  = 0c657d851605d0aa284f38df34bd02d4c2c3bb03246ac3d89c49e6355841d124
U[1][0]  = c2ce188ea722d7bc1c7645e449bf3fa4745af39e194d82138c51188b5a18b872
U[1][1]  = 5a8456b909be71ecc6f63865f9a14e3556e2e3f2509a786de06774b868c6290b
a_final  = ad0ab50f80e5607a30b0449ba12421c3aab3d55dd18947d8255208a0e73e2f06
b_final  = 7c9e9a9ac74740e85ed173edf096d472e36e0a65a55653519225518934385302

```

11.12 eCP Sub-Proof ($k = 2$, $d = 2$, $m = 1$)

Parsed per Section 4.5. The cross-term pairs $[A^{(0)}_{j,\ell},\ A^{(1)}_{j,\ell}]$ enter Algorithm 5 (Section 10) and the Mega-MSM (Section 8.6.2). With $k = 2$ there are $2(k-1) = 2$ pair indices per round; each pair consists of a row-0 point (`[0]`) and a row-1 point (`[1]`). Cross-terms enter the Mega-MSM as described in §11.14.

```
A[0][0][0] = 3064e4383decb5be08b91ac267b9ec89ed562f6f1c78f3c2b64e015ae0e7a74f
A[0][0][1] = fe5086015939952941fe5342c1e1cc95c80487409ffbdb91ac7211fdf0a4327e
A[0][1][0] = c26e9e6c9b81f8b2eeddb5ac94e7f5d09460a85990d82742e10f42a27a34e119
A[0][1][1] = a0be5451dd752442a743298bee2aa9a5d264b87515b11a42d3789b821c975110
A[1][0][0] = d6b3280ff8b9cc0d97410eb51de1b37ec06783733ba86a813320952078546e16
A[1][0][1] = 442d2e6fd7ccaca45a8ca70001309de46cbea3554860ff23155861bc99b6aa7c
A[1][1][0] = f6c9c2c14abde8076666c25b55e37bbdaf3662e8f06f7dd9ce4b14847c312452
A[1][1][1] = 461fe4e5a3c5f8f9393d8bc9bbd2f406381f13cbbd3add5d358a259113d63f71
z_final    = b763423ee5460a4107d8ef4217b0a5884b95d90b333c11b9c30f11ab07da5b00
```

11.13 IPA Scalar Expansion

The IPA module produces the following scalars from the cross-term challenges c_j (one per folding round) and the base vectors $(\mathbf{a}_{\text{final}}, \mathbf{b}_{\text{final}})$. For $d = 2$, $k = 2$, $m = 1$: there is one base element ($\texttt{a\_final} = \texttt{sg}[3]$, $\texttt{b\_final} = \texttt{sh}[0]$), and s_P is the product $\prod_j c_j^{k-1}$ over the two rounds.

```
sP     = 25e814629f682fa7b8780024f1d3583707b0bcaad9374c8623e71b6a8665dc06
sQ     = c8b1ba2b23a2f3d8d5ee1747a0fac8e782b3286f5da36f7cb77454a5938cab0e
sg[0]  = 18fb9a6cb91e7b943fca8abca14b6215fb5f484c330238da6082bc2237b68c07
sg[1]  = 0d9c88a083671b56a691195cdd97d2be3236589d8bd9ad3b11c4a6480a09dd04
sg[2]  = e6e9c414b48023a678cc2fd822a362cc6303679c1df9fcf51ce72d4c4747f706
sg[3]  = ad0ab50f80e5607a30b0449ba12421c3aab3d55dd18947d8255208a0e73e2f06
sh[0]  = 7c9e9a9ac74740e85ed173edf096d472e36e0a65a55653519225518934385302
sh[1]  = 75bfadda670e21b5595a69e563c898e198c8628cb77bb4f1a074d78516691c06
sh[2]  = efffd8a5ee7ccf38d0f42ae66bceadc9dd36cca038fa124084c0d5b8f2b80905
sh[3]  = 39f3d279f6aac9a89d0b6643758852b5798aa562287ecbb8b5f485d04d105301

```

The IPA cross-term scalars $\texttt{sU}[r \cdot 2(k-1) + \ell]$ are indexed in flattened row-major order (round r, sub-index ℓ):

```
sU[0]  = feedab45eeacbfac9f5a8fc904755cb82f1b919870527c4378bb6c57de33ce00
sU[1]  = 5402af86cf11e8c1953c246e69e3b1473cc93f57234dfd8c5ada4a062446d004
sU[2]  = 0100000000000000000000000000000000000000000000000000000000000000
sU[3]  = 7e0dfd78e05f0469181e2b5a06171bbf74db656e68e35b42bf6c94416d81eb0f
```

Note: $\mathtt{sU}[2] = 1$ (the scalar one in little-endian encoding) because at round $r = 1$, sub-index $\ell = 0$ the contributing product reduces to 1 after the two folding steps for this specific challenge. Each cross-term point $U[r][\ell]$ enters the Mega-MSM with scalar $-\mathtt{sU}[r \cdot 2(k-1) + \ell]$.

11.14 eCP Scalar Expansion

The eCP module produces the following scalars. $s_{P,e}$ (or `sP_ecp`) is the product $\prod_j c_j^k$ (exponent k, not $k-1$ as in IPA). $\mathtt{zs}[i]$ is the expanded coefficient for generator $g[i]$ (analogous to `sg`); it also applies to $C_1'[i]$ and $C_2'[i]$ in the Mega-MSM (see §11.16).

```
sP_ecp = 9cd143250c751e9884d87e24f45dbbeb57b07191a97cd566b30d9fcfa1d16b04
zs[0]  = 09d7395e255ad762ac5ab20d6b77a9620442f362ad244cae2c787650b487dc0d
zs[1]  = af353d7a429d4f23ee569eef00e0011348295cee0bf20aecf49f8c0c66570003
zs[2]  = 3ae1497eb1340e2622c9309bd6f87e55b8a9a2702a188406a02fc0563414c30f
zs[3]  = d26c240e738ab92fd9ad6a2c46d659209977fd195e7565fbcc6b067c7f1faf09
```

The eCP cross-term scalars $\mathtt{sA}[r \cdot 2(k-1) + \ell]$ (row-major, same indexing as `sU`):

```
sA[0]  = a20ab289830b5acc01f3cd4dd5c4fc6c1edc52692b1403601970f2359772380f
sA[1]  = 17f420305a616ad7161ce2f0088702f649507aca77f37eef3dbf25285d5c220a
sA[2]  = 8225c78cc04e07b27a56893be2da7ae8b4391b11eef60fd55079f4e657d44600
sA[3]  = 23dfb533dc7c58da8cb6df48e4fab0da64afbc65dd755be4ba31acb1fa27a409
```

Each cross-term pair $(A[r][\ell][0],\ A[r][\ell][1])$ enters the Mega-MSM with scalars $(-\mathtt{sA}[i] \cdot r^4,\ -\mathtt{sA}[i] \cdot r^3)$ respectively, where r is the batching scalar `batching_r` from §11.8.

11.15 Generator Scalars

The Mega-MSM scalar for each generator pair $(g[i],\ h[i])$ is computed as follows:

$$\mathtt{gscalar}[i] = \mathtt{sg}[i] - x \cdot y^{-i} \cdot \mathtt{wR}[i] \cdot s_P, \tag{30}$$

$$\mathtt{hscalar}[i] = y^{-i}\,\mathtt{sh}[i] + \big(y^{-i}(x\,\mathtt{wL}[i] + \mathtt{w0}[i]) - 1\big)(-s_P) + y^{-i}\,x_{\text{ipp}}(-s_P)\,\mathtt{wV}[i]. \tag{31}$$

This is Reading A of the formula (parenthesization around $y^{-i}(\cdot) - 1$). Note the *final* g-scalar in the Mega-MSM is $\mathtt{gscalar}[i] + \mathtt{zs}[i] \cdot r^4$ (the eCP contribution is added at assembly time; see §11.16). Compare after implementation step 7.

```
gscalar[0] = 699e3cce7867985f1687490b01c66a669f5aa53bb503bb552eae6b185bb55b0c
gscalar[1] = 608a8c4a3514bb578426035fe7e66adbaa2be543d904437d2031e433484e4b0f
gscalar[2] = fad95863c81d8b6e403b2b69dc9ed52ff42847fffd6ccc5d4e60b7ba93eaf408
gscalar[3] = ad0ab50f80e5607a30b0449ba12421c3aab3d55dd18947d8255208a0e73e2f06
hscalar[0] = 51c0b535eea8d231a5ed02bc6537a5803e8cc33a154201688fc685984bb3430e
hscalar[1] = fa413a0ae99976eff1c3c05b9826f2a22d8ec67f5e1b12c6416f7df32ff0380d
hscalar[2] = 42c67db3701d5c7bee0166bff2f2e39da52579e895b586055e1eb41be13f4907
hscalar[3] = 5cb64e18d2a08fae1d0d615cdaf88db2bb4a4c0aa736c597623b8da7d8beca09
```

11.16 Mega-MSM Key Scalars

The following intermediate scalars resolve the formula ambiguities most likely to confuse an implementer:

(A) w_{agg} distribution. The E_0 rows A_I, A_O, S, $g[i]$ (before adding eCP term), $h[i]$, and $U[r][\ell]$ do *not* carry w_{agg}. The factor w_{agg} appears *only* in the B scalar B_s, multiplying s_Q and the expected inner product.

(B) B_s formula. The exact grouping is:

$$B_s \;=\; w_{\text{agg}}(s_Q - \hat{t}\, s_P) \;+\; r\big(x^2(w_c + \delta) - t_x\big) \;-\; r^2\, t_{c,x}, \tag{32}$$

where $\hat{t} = t_x + x_{\text{ipp}}\, t_\times + x_{\text{ipp}}^2\, t_{c,x}$ is the expected inner product. The first term is the E_0 contribution; r- and r^2-terms are from E_1 and E_2 respectively.
The Mega-MSM also requires:

$$B_b \;=\; x_{\text{ipp}}\, \mu_c\, s_P + \mu\, s_P - r^2\, \tau_{c,x} - r\, \tau_x, \tag{33}$$

with the final B-point scalar being $B_s + r^3 s_{P,e}\, \rho_c$ and the H-point scalar being $B_b + r^4 s_{P,e}\, \mu_c$.

```
delta = 63ce2519f015735fb441df0cf37aef34172fe39c0ffa26bb440d8a052b6e8d0f
Bs    = baf73f0663a8dc723571bd01dc066771e1c1643fe133200150f60de3e9b6ff03
Bb    = baf6c661a8d714cb0f60073cd455fdca141e9d968b0308d9382d4d0644dd3606

```

Summary of Mega-MSM point-to-scalar mapping (all other points not listed below use the formulas in Section 8.7 directly with values from §11.8–11.14):

Point	Scalar expression	Section
A_I	$-x \cdot s_P$	8.7
A_O	$-x^2 \cdot s_P$	8.7
S	$-x^3 \cdot s_P$	8.7
V	$-x_{\text{ipp}} s_P - r^4 s_{P,e}$	8.7
S'	$-x_{\text{ipp}}\, x'\, s_P \;-\; r^4 x'\, s_{P,e}$	8.7
B	$B_s + r^3 s_{P,e} \rho_c$	Eq. (32)
H	$B_b + r^4 s_{P,e} \mu_c$	Eq. (33)
F	$r^3 \mathsf{ch} \cdot s_{P,e} \rho_c$	8.7
$g[i]$	$\mathtt{gscalar}[i] + \mathtt{zs}[i] \cdot r^4$	§11.15
$h[i]$	$\mathtt{hscalar}[i]$	§11.15
T_1'	$r^2 x'$	8.7
T_2	$r^2 + r x^2$	8.7
T_j $(j \neq 2)$	$r x^j$	8.7
S_1'	$r^3 x'(-s_{P,e})$	8.7
S_2'	$r^3 \mathsf{ch}\, x'(-s_{P,e})$	8.7
$C_1[i]$	$r^3(-s_{P,e}) e[i]$	8.7
$C_2[i]$	$r^3 \mathsf{ch}(-s_{P,e}) e[i]$	8.7
$C_1'[i]$	$\mathtt{zs}[i] \cdot r^3$	§11.14
$C_2'[i]$	$\mathtt{zs}[i] \cdot r^3 \cdot \mathsf{ch}$	§11.14
$A[r][\ell][0]$	$-\mathtt{sA}[2r + \ell] \cdot r^4$	§11.14
$A[r][\ell][1]$	$-\mathtt{sA}[2r + \ell] \cdot r^3$	§11.14

(ch denotes `chall_batched_ecp` from §11.8; r denotes `batching_r`.) Note T_2 appears *twice* in the MSM (once from the linearisation check with scalar r^2 and once from the polynomial check with scalar rx^2); implementers must add both contributions.

11.17 Final Verification

The Mega-MSM result (Verification 6, Section 8.7):

```
mega_check  = 0000000000000000000000000000000000000000000000000000000000000000
is_identity = true

```

The compressed Mega-MSM result is 32 zero bytes (the Ristretto255 identity encoding, Section 3.1), so IsIdentity(R) = true and the proof is Accepted.

References

[1] Thi Van Thao Doan, Olivier Pereira, and Thomas Peters. Maya: A short shuffle argument with fast verification. Cryptology ePrint Archive, Paper 2026/941, 2026.

[2] Thi Van Thao Doan, Olivier Pereira, and Thomas Peters. MAYA: A Short Shuffle Argument With Fast Verification. `https://github.com/uclcrypto/maya_shuffle_proof`, 2026.

[3] Douglas Wikström. A commitment-consistent proof of a shuffle. In *Australasian Conference on Information Security and Privacy*, pages 407–421. Springer, 2009.

[4] Jonathan Bootle, Andrea Cerulli, Pyrros Chaidos, Jens Groth, and Christophe Petit. Efficient zero-knowledge arguments for arithmetic circuits in the discrete log setting. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 327–357. Springer, 2016.

[5] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *2018 IEEE symposium on security and privacy (SP)*, pages 315–334. IEEE, 2018.

[6] Thomas Attema. Compressed σ-protocol theory. 2023.

[7] Max Hoffmann, Michael Klooß, and Andy Rupp. Efficient zero-knowledge arguments in the discrete log setting, revisited. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pages 2093–2110, 2019.

[8] Ivan Damgård. On σ-protocols. *Lecture Notes, University of Aarhus, Department for Computer Science*, 84:84–105, 2002.

[9] Jacob T. Schwartz. Fast probabilistic algorithms for verification of polynomial identities. *Journal of the ACM*, 27(4):701–717, 1980.

A Ristretto255 Constants

This appendix lists the hexadecimal constants needed to hard-code the Ristretto255 parameter set. All values are in little-endian byte order.

Curve equation. Curve25519: $y^2 = x^3 + 486662x^2 + x$ over $\mathbb{F}_q$ with $q = 2^{255} - 19$.

Scalar field order. $p = 2^{252} + 27742317777372353535851937790883648493$:

```
0x10000000 00000000 00000000 00000000
14def9de a2f79cd6 5812631a 5cf5d3ed
```

Basepoint (canonical compressed encoding).

```
e2f2ae0a 6abc4e71 a884a961 c500515f
58e30b6a a582dd8d b6a65945 e08d2d76
```

(32 bytes, little-endian.)

Identity. 32 zero bytes (`0x00...00`).

B Label Reference and Transcript Schedule

This appendix collects every normative string label used in the MAYA protocol for quick reference. These labels enter hash computations. Any deviation produces different outputs and breaks verification. The labels match the reference implementation [2] exactly.

For the complete step-by-step transcript operations in execution order, see:

- Main transcript: Section 6.4 (schedule steps #0–#33);
- Challenge vector: Algorithm 3 (Section 7);
- IPA sub-transcript: Algorithm 4 (Section 9);
- eCP sub-transcript: Algorithm 5 (Section 10).

B.1 CRS Labels

Label (bytes)	Used in	Purpose
`"PedersenGens_blinding"`	Algorithm 1	Domain separation for B_{blinding}
`"PedersenGens_F"`	Algorithm 1	Domain separation for generator F
`0x01`, `0x02`	Algorithm 1	Counter byte (first/second 32-byte half)
`"GeneratorsChain"`	Algorithm 2	HMAC key for seed derivation $(\boldsymbol{g}, \boldsymbol{h})$
`[0x47, 0, 0, 0, 0]`	Algorithm 2	Seed message: type `G` + party index 0
`[0x48, 0, 0, 0, 0]`	Algorithm 2	Seed message: type `H` + party index 0
`"bp_"` $\|\|\mathsf{b}_4(i)\|\|$`0x01`	Algorithm 2	First half of generator i
`"bp_"` $\|\|\mathsf{b}_4(i)\|\|$`0x02`	Algorithm 2	Second half of generator i

B.2 Challenge Vector Labels

Label	Operation	Purpose
`"FiatShamir-e"`	Init	Separate transcript for $\boldsymbol{e}$
`"challenge-e-derivation"`	Append(`"dom-sep"`, $\cdot$)	Domain separator
`"pk_g"`, `"pk_f"`, `"ck_h"`	CommitPoint	Public key elements
`"ck_n"`	CommitU64	Generator capacity N
`"ck_g_seed"`, `"ck_h_seed"`	Append	CRS seeds (Algorithm 2)
`"c1"`, `"c2"`, `"c1p"`, `"c2p"`	Append	Ciphertext coordinate vectors (one call, $32n$ bytes each)
`"e_seed"`	ChallengeBytes32	PRG seed (32 bytes)
`"e_"` $\|\|\mathsf{b}_4(i)\|\|$`0x01`	H (key = seed)	First half of e_i
`"e_"` $\|\|\mathsf{b}_4(i)\|\|$`0x02`	H (key = seed)	Second half of e_i

B.3 IPA Sub-Transcript Labels

The IPA sub-protocol appends to the same main transcript. The full algorithm is in Section 9.

Label	Data	Operation	Purpose
`"protocol-name"`	`"ipa_fold"`	Append	Protocol identifier
`"n"`	$\mathsf{b}_8(n_{\text{circuit}})$	Append	Padded dimension
`"k"`	$\mathsf{b}_8(k)$	Append	Folding factor
Per round j, per index $\ell = 0, \ldots, 2k-3$:			
`"U_round"`	$\mathsf{b}_8(j)$	Append	Round index
`"U_index"`	$\mathsf{b}_8(\ell)$	Append	Cross-term index
`"U_point"`	$U_{j,\ell}$	CommitPoint	Cross-term point
After all cross-terms of round j:			
`"challenge_prefix"`	`"c_"`	Append	
`"challenge_index"`	$\mathsf{b}_8(j)$	Append	
`"challenge_separator"`	—	ChallengeScalar	$\Rightarrow c_j$

B.4 eCP Sub-Transcript Labels

The eCP sub-protocol also appends to the same main transcript. The full algorithm is in Section 10.

Label	Data	Operation	Purpose
`"protocol-name"`	`"ecp_fold"`	Append	Protocol identifier
`"n"`	$\mathsf{b}_8(n_{\text{circuit}})$	Append	Padded dimension
`"k"`	$\mathsf{b}_8(k)$	Append	Folding factor
Per round j, per pair $\ell = 0, \ldots, 2k-3$:			
`"A_round"`	$\mathsf{b}_8(j)$	Append	Round index
`"A_index"`	$\mathsf{b}_8(\ell)$	Append	Pair index
`"A_point_0"`	$A_{j,\ell}^{(0)}$	CommitPoint	Row-0 point
`"A_point_1"`	$A_{j,\ell}^{(1)}$	CommitPoint	Row-1 point
`"challenge_prefix"`	`"c_"`	Append	
`"challenge_index"`	$\mathsf{b}_8(j)$	Append	
`"challenge_separator"`	—	ChallengeScalar	$\Rightarrow c_j$

B.5 Consolidated Label Inventory

Every transcript label string in the protocol, its context, operation type, and semantic meaning. Labels are byte-exact; two rows sharing the same string (e.g., `"n"`, `"k"`, `"dom-sep"`, `"challenge_separator"`) are *distinct uses in distinct contexts* and are disambiguated by transcript position, never by renaming.

Label	Context	Operation	Meaning
"MAYA-Shuffle-v1"	main	Init	protocol/version identifier
"FiatShamir-e"	e-vector	Init	dedicated e transcript
"dom-sep"	main (#1, #3); e-vector	Append	domain separators
"circuit_n"	main #2	CommitScalar	padded dimension n_{circuit}
"V"	main #4	CommitPoint	vector commitment
"m"	main #5	CommitU64	n_{circuit} (u64 form)
"shuffle challenge"	main #6	ChallengeScalar	u
"A_I", "A_O", "S"	main #7–9	CommitPoint	wire commitments
"y", "z"	main #10–11	ChallengeScalar	y, z
"T_1", "T_3"–"T_6", "T_2"	main #12–17	CommitPoint	poly. coeffs (T_2 last)
"x"	main #18	ChallengeScalar	x
"t_x", "t_x_blinding", "e_blinding"	main #19–21	CommitScalar	t_x, τ_x, μ
"S_prime", "T_1_prime", "S1_prime", "S2_prime"	main #22–25	CommitPoint	Phase-2 commitments
"x_prime"	main #26	ChallengeScalar	x'
"tc_x", "tc_x_blinding", "ec_blinding"	main #27–29	CommitScalar	$t_{c,x}, \tau_{c,x}, \mu_c$
"r_blinding", "t_cross"	main #30–31	CommitScalar	$\rho_c, t_\times$
"x_ipp", "w_agg"	main #32–33	ChallengeScalar	$x_{\text{ipp}}, w_{\text{agg}}$
"protocol-name"	IPA / eCP	Append	"ipa_fold" / "ecp_fold"
"n"	IPA / eCP	Append	$b_8(n_{\text{circuit}})$
"k"	IPA / eCP	Append	$b_8(k)$ (folding factor)
"U_round", "U_index", "U_point"	IPA	Append/CommitPoint	cross-term schedule
"A_round", "A_index", "A_point_0", "A_point_1"	eCP	Append/CommitPoint	cross-term pairs
"challenge_prefix" ("c_"), "challenge_index"	IPA / eCP	Append	round tag
"challenge_separator"	IPA / eCP	ChallengeScalar	round challenge c_j/c_j^e
"chall_batched_ecp"	main #34	ChallengeScalar	α_{ecp}
"batching_r"	main #35	ChallengeScalar	r
"pk_g", "pk_f", "ck_h", "ck_n"	e-vector	CommitPoint/U64	statement binding
"ck_g_seed", "ck_h_seed"	e-vector	Append	CRS seeds
"c1", "c2", "c1p", "c2p"	e-vector	Append	ciphertext vectors ($32n$ B)
"e_seed"	e-vector	ChallengeBytes32	PRG seed z_e
"chal", "chal_ext", "chal_update"	all	internal (Eqs. 12–15)	challenge machinery
"PedersenGens_blinding", "PedersenGens_F"	CRS	HMAC msg	Alg. 1
"GeneratorsChain", "bp_"	CRS	HMAC key/msg	Alg. 2
"e_"	e-vector	HMAC msg	Alg. 3 Stage 2

C FromUniformBytes: Elligator2 Map for Ristretto255

This appendix provides the step-by-step algorithm for the FromUniformBytes function used in Sections 5.1 and 5.2. It follows RFC 9496, §3.3.2 (one_way_map) and §3.2.4 (MAP), and operates on Curve25519 field arithmetic with $q = 2^{255} - 19$.

Most implementers will not need this appendix: the function is available in standard Ristretto255 libraries (Rust: curve25519-dalek; Go: filippo.io/edwards25519; Python: ristretto255; JavaScript: ristretto255-js). This appendix is provided for completeness, for implementers writing against a language or curve without library support, or for auditors verifying CRS deriva-

tion.

Constants

All arithmetic is in $\mathbb{F}_q$ with $q = 2^{255} - 19$.

$$d = -\tfrac{121665}{121666} \pmod q \qquad \text{(Edwards curve constant)}$$
$$\sqrt{-1} = 2^{(q-1)/4} \pmod q$$
$$\mathsf{SQRT\_AD\_MINUS\_ONE} = \sqrt{a \cdot d - 1} \pmod q \quad \text{where } a = -1$$
$$\mathsf{INVSQRT\_A\_MINUS\_D} = (\sqrt{a - d})^{-1} \pmod q$$
$$\mathsf{ONE\_MINUS\_D\_SQ} = 1 - d^2 \pmod q$$
$$\mathsf{D\_MINUS\_ONE\_SQ} = (d-1)^2 \pmod q$$

Step 1: Split and Decode

Given 64 input bytes $b_0, \ldots, b_{63}$:

1. $r_0 \leftarrow b_0 \| \cdots \| b_{31}$, interpreted as a 256-bit LE integer, reduced modulo q.
2. $r_1 \leftarrow b_{32} \| \cdots \| b_{63}$, interpreted as a 256-bit LE integer, reduced modulo q.

Step 2: Elligator2 Map ($\mathsf{MAP} : \mathbb{F}_q \to$ Edwards point)

For each $r \in \{r_0, r_1\}$, compute (X, Y, Z, T) in extended coordinates on the Edwards curve $-x^2 + y^2 = 1 + d \cdot x^2 y^2$:

1. $r_{\text{sq}} \leftarrow \sqrt{-1} \cdot r^2$
2. $u \leftarrow (r_{\text{sq}} + 1) \cdot \mathsf{ONE\_MINUS\_D\_SQ}$
3. $v \leftarrow (-1 - r_{\text{sq}} \cdot d) \cdot (r_{\text{sq}} + d)$
4. $(\mathit{was\_sq}, s) \leftarrow \mathsf{sqrt\_ratio\_i}(u, v)$
5. $s' \leftarrow -|s \cdot r|$
6. $s \leftarrow s'$ if $\neg\, \mathit{was\_sq}$; otherwise s unchanged
7. $c \leftarrow -1$ if $\neg\, \mathit{was\_sq}$; otherwise $c \leftarrow r_{\text{sq}}$
8. $N \leftarrow c \cdot (r_{\text{sq}} - 1) \cdot \mathsf{D\_MINUS\_ONE\_SQ} - v$
9. $w_0 \leftarrow 2sv, \quad w_1 \leftarrow N \cdot \mathsf{SQRT\_AD\_MINUS\_ONE}, \quad w_2 \leftarrow 1 - s^2, \quad w_3 \leftarrow 1 + s^2$
10. Return $(X, Y, Z, T) = (w_0 w_3,\ w_2 w_1,\ w_1 w_3,\ w_0 w_2)$

Step 3: Combine

$P_0 \leftarrow \mathsf{MAP}(r_0), \quad P_1 \leftarrow \mathsf{MAP}(r_1)$. Return $P_0 + P_1$.

Auxiliary: $\mathsf{sqrt\_ratio\_i}(u, v)$

Returns $(\mathit{was\_square}, s)$ where $s^2 = u/v$ if a square root exists, or $s^2 = \sqrt{-1} \cdot u/v$ otherwise.

1. $v_3 \leftarrow v^2 \cdot v; \quad v_7 \leftarrow v_3^2 \cdot v$
2. $r \leftarrow (u \cdot v_3) \cdot (u \cdot v_7)^{(q-5)/8}$
3. $c \leftarrow v \cdot r^2$
4. $\mathit{ok} \leftarrow (c = u); \quad \mathit{flip} \leftarrow (c = -u); \quad \mathit{flip\_i} \leftarrow (c = -u\sqrt{-1})$
5. If $\mathit{flip} \vee \mathit{flip\_i}$: $r \leftarrow r \cdot \sqrt{-1}$
6. $r \leftarrow |r|$ (negate if odd)
7. $\mathit{was\_square} \leftarrow \mathit{ok} \vee \mathit{flip}$
8. Return $(\mathit{was\_square}, r)$

Sign convention. A field element $s \in \mathbb{F}_q$ is *negative* if its canonical LE encoding has the least significant bit set (i.e. s is odd as an integer in $[0, q)$). $|s|$ returns s if non-negative, $q - s$ otherwise.

Exponentiation. $(q - 5)/8 = 2^{252} - 3$ is a fixed 253-bit integer. Implementations should use a constant-time exponentiation chain.